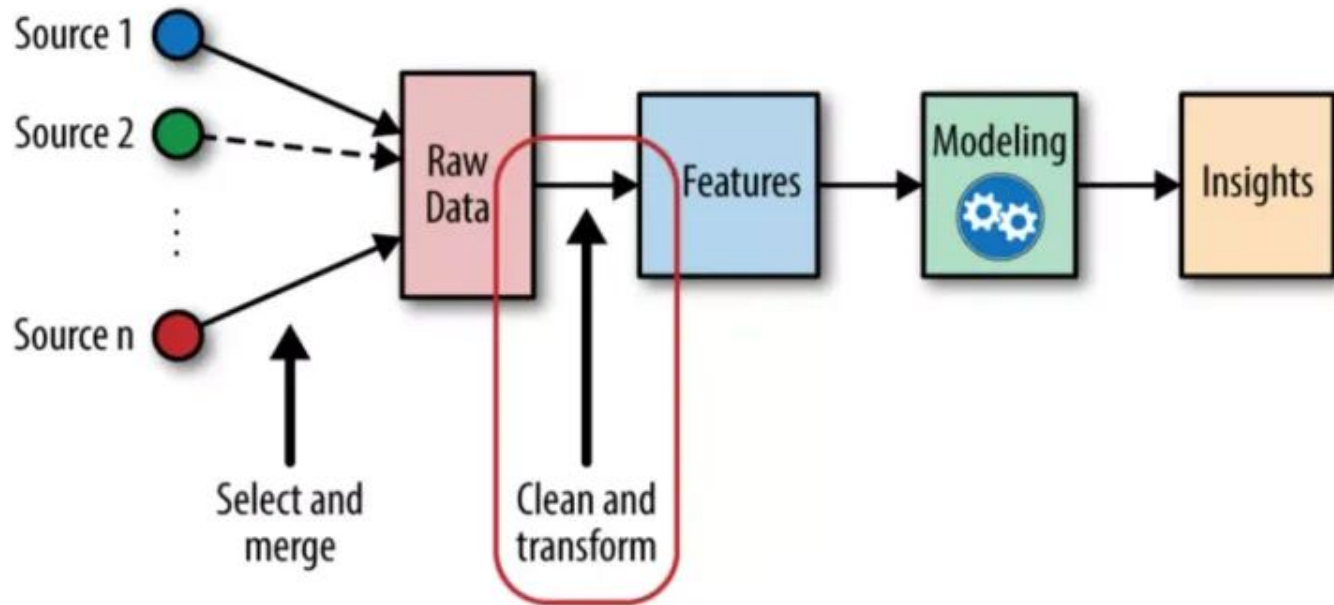


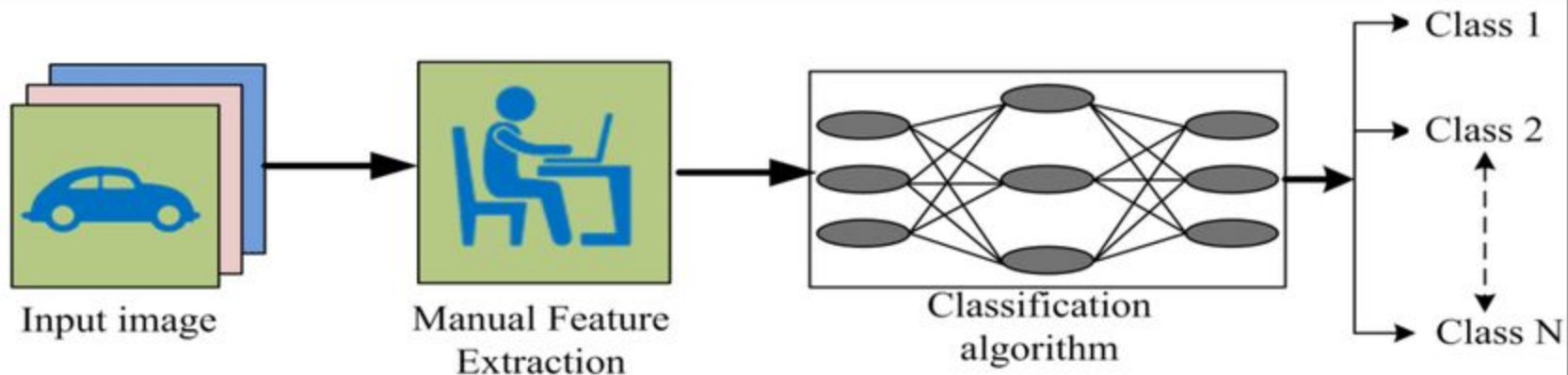
Sequential Models

Basics of DL and RNN



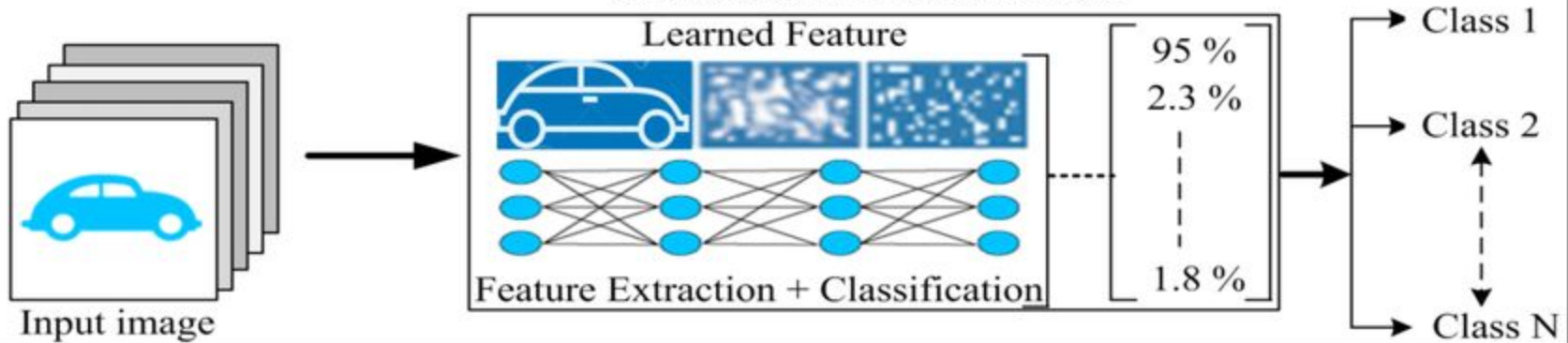
feature engineering goes here!

(a) Machine learning



(b) Deep learning

Convolutional Neural Network



Task

Predict whether a student will pass an exam

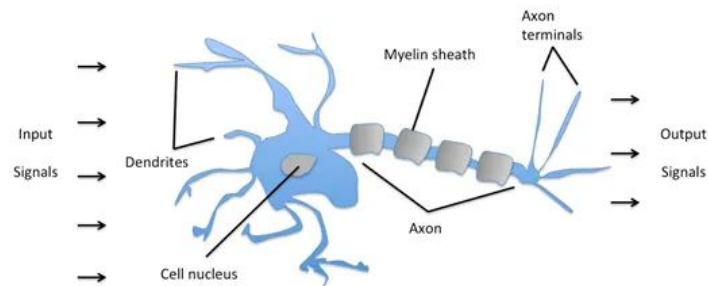
How you will solve this problem using ML and DL:

1- feature selection

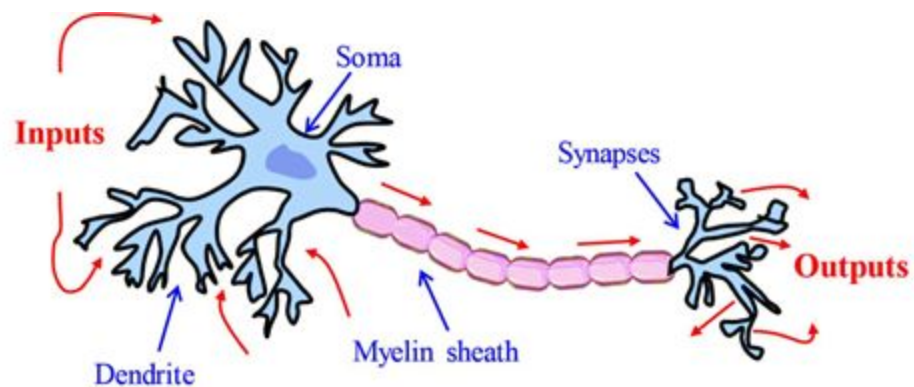
2- which model

ANN

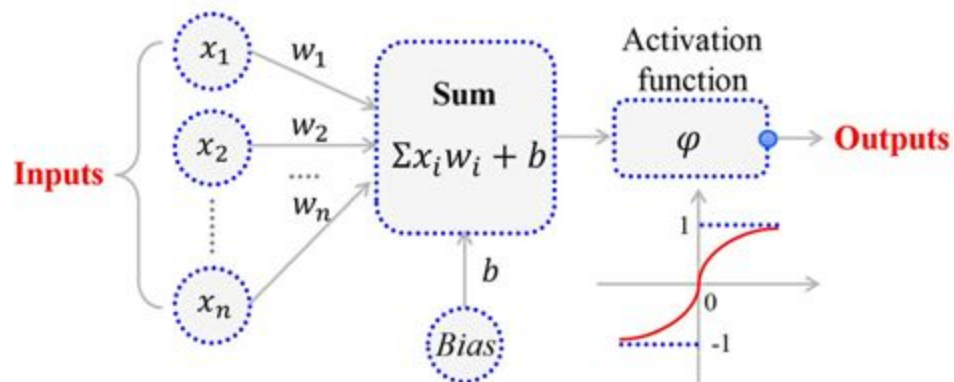
- Brain-inspired system designed to learn patterns like humans
- Consists of input, hidden, and output layers
- Hidden layers transform input into meaningful features
- Excellent for recognizing complex, nonlinear patterns
- Early “perceptrons” existed since the 1940s
- Became powerful after backpropagation, enabling networks to learn from errors
- Deep learning uses many hidden layers to extract features and achieve high accuracy



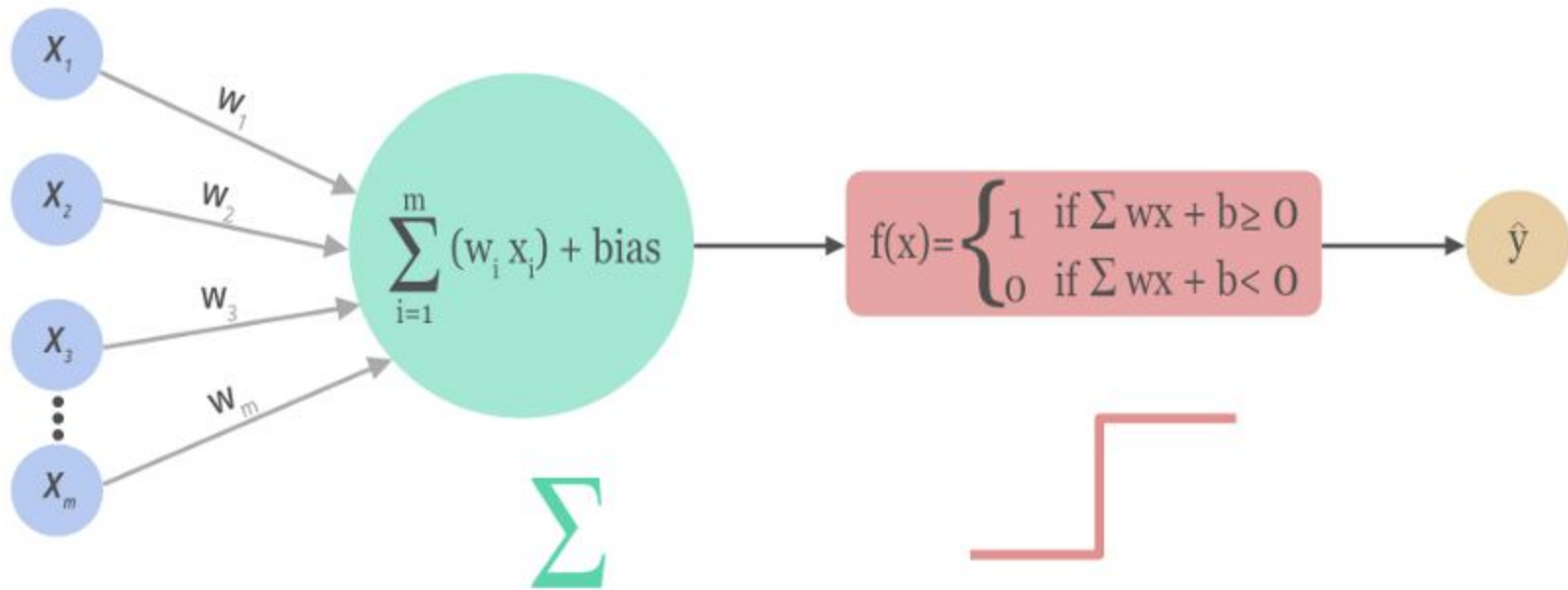
Schematic of a biological neuron.



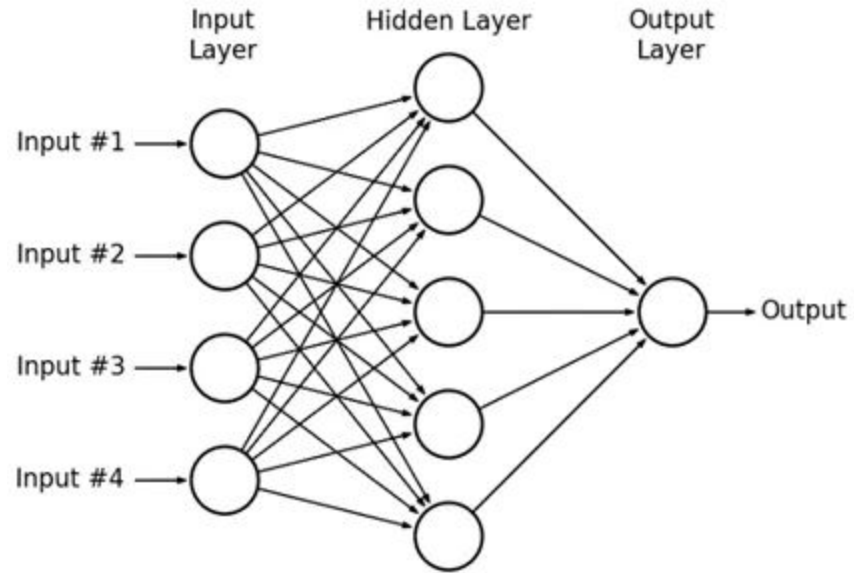
(a) Biological neuron.



(b) Artificial neuron.



Basic Structure of ANN



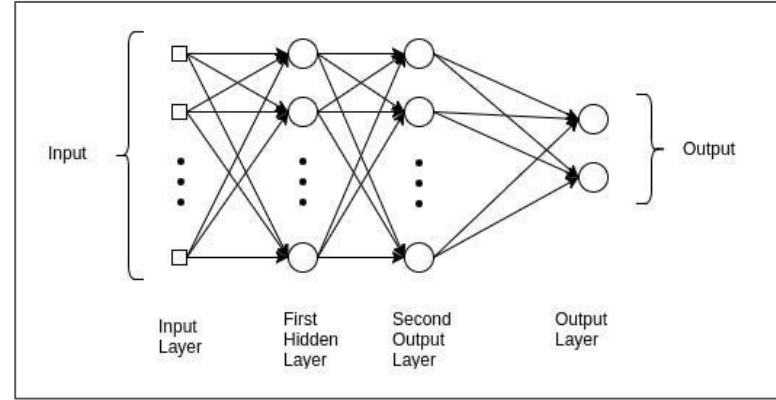
Types of ANN

1- FeedForward ANN:

Feedforward models are neural networks where the information flows in one direction—from input to output—without any cycles or loops. Here are some common examples:

examples :

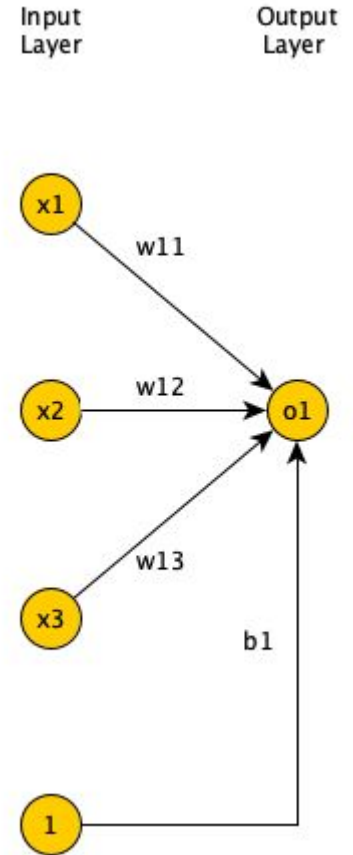
- **Multilayer Perceptrons(MLP)**
 - The classic feedforward neural network with one or more hidden layers.
- **Convolutional Neural Network(CNN)**
 - Specialized feedforward networks for processing grid-like data such as images.



Feedforward neural networks (FNN)

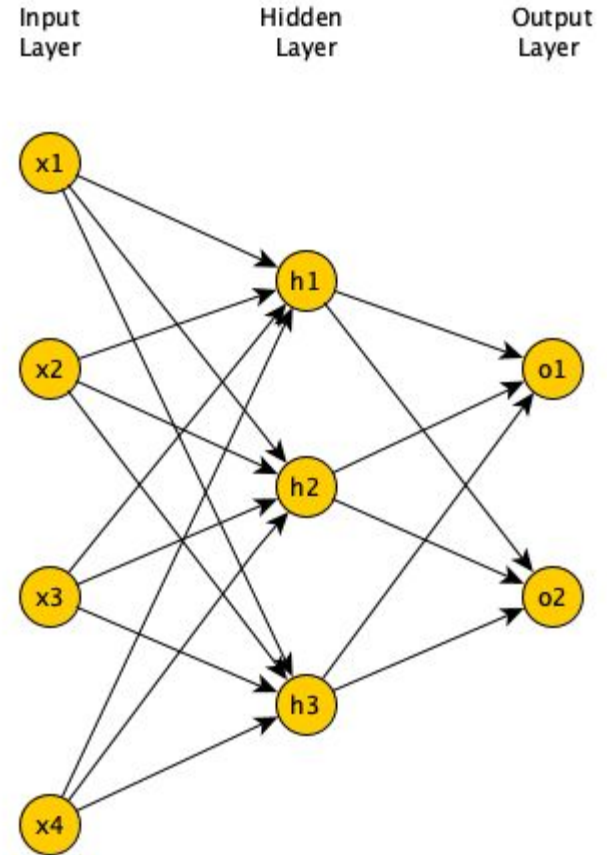
In feedforward neural networks (FNN) a single training example is presented to the network, after which the network generates an output. For example, a lung X-ray image is passed to a FNN, and the network predicts tumor or no tumor.

Example of Single layer FNN



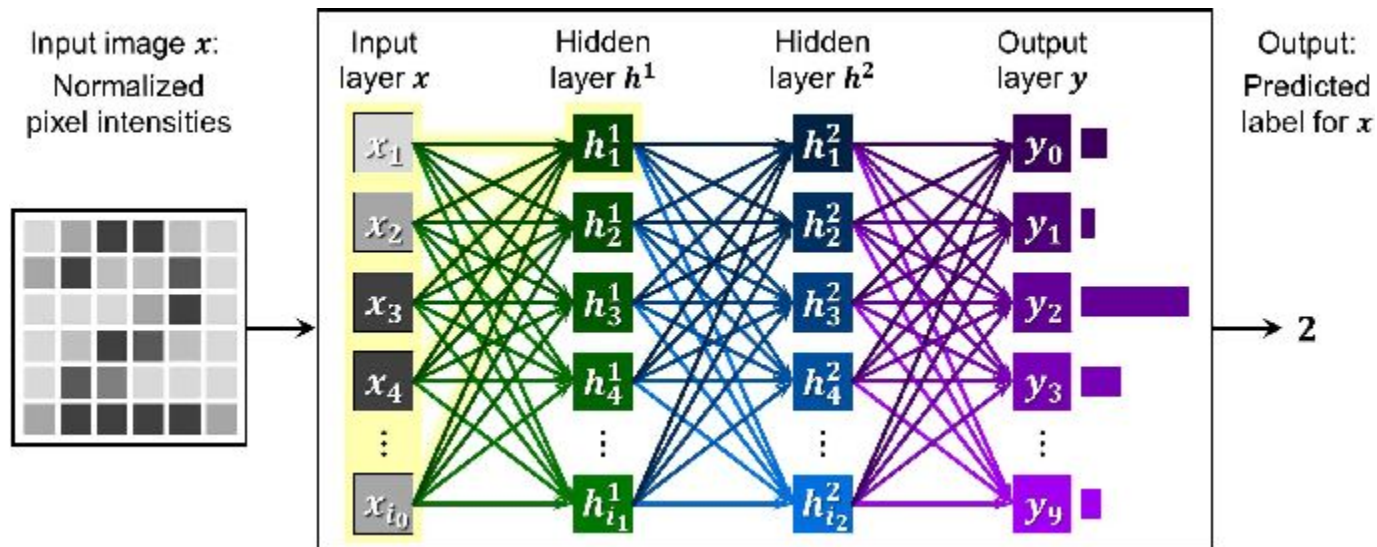
Multi-layer FNN

A single layer FNN cannot solve problems in which the data is not linearly separable, such as the XOR problem. Adding one (or more) hidden layers to FNN enables it to solve problems in which data is non-linearly separable.



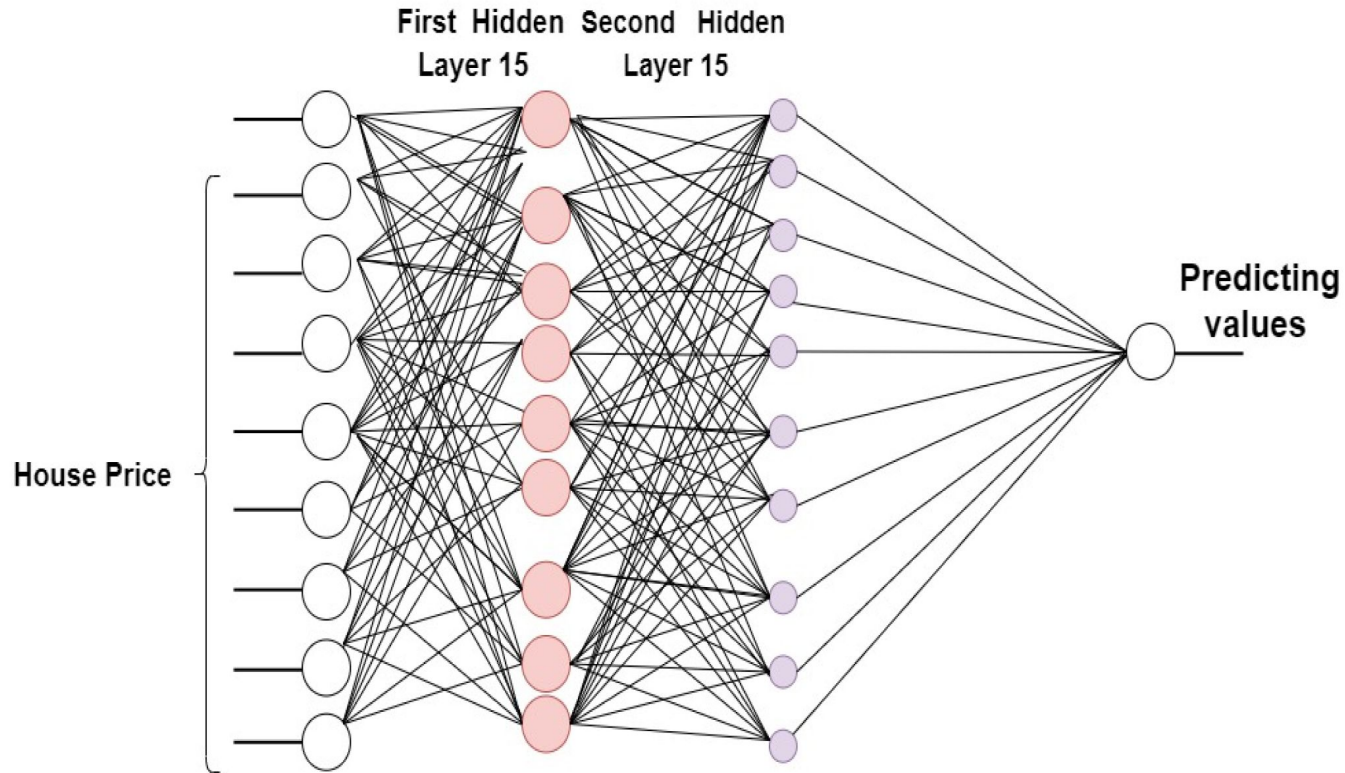
Which model ?

Regression/prediction or classification?



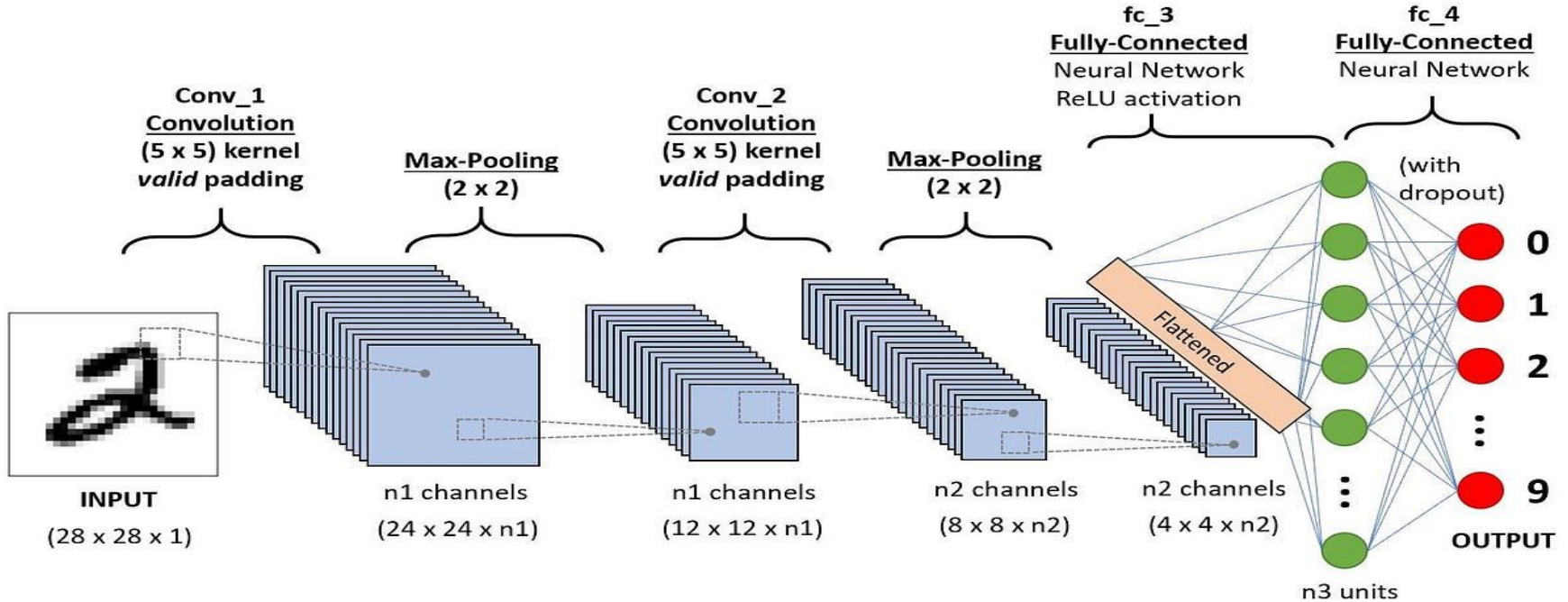
Which model ?

Regression/prediction or classification?

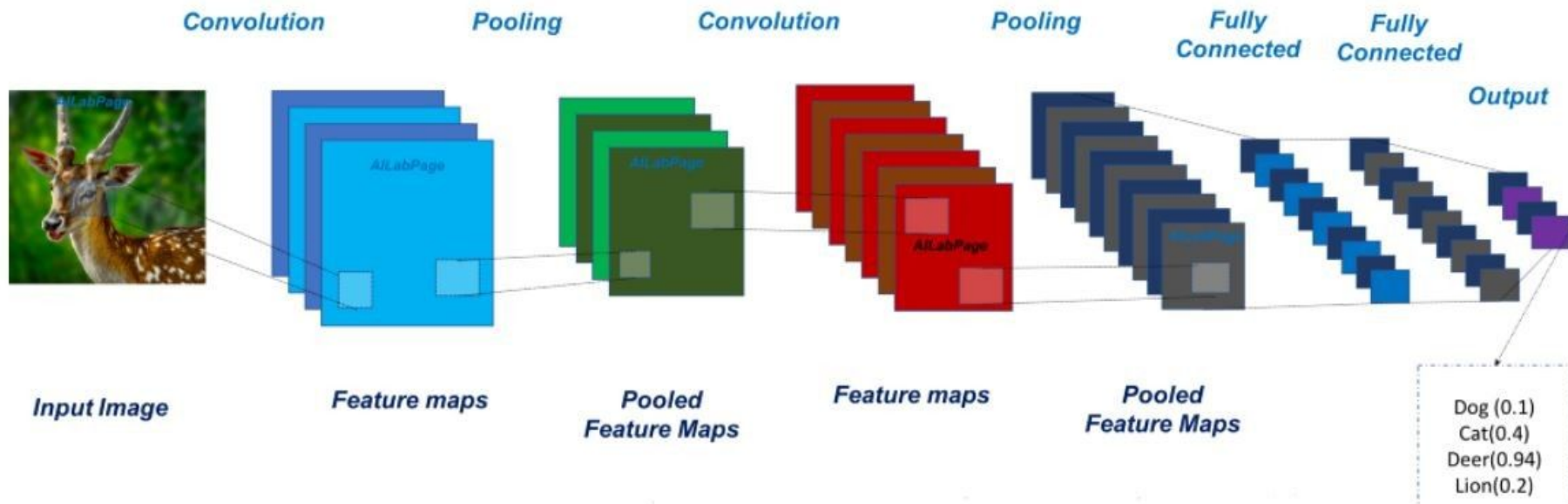


Which model ?

Regression or classification?

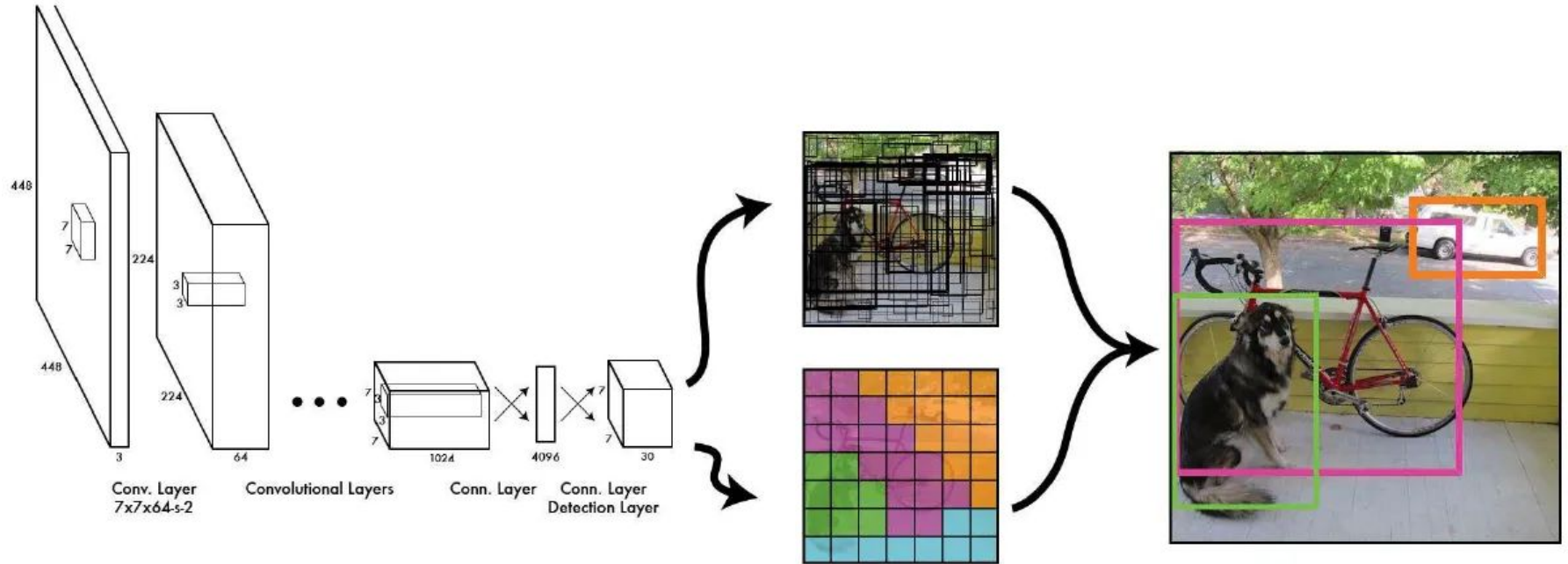


ImageNet for image classification



You Only Look Once(YOLO)

specifically designed for real-time object detection



ASIA

INDIA PAKISTAN PHILIPPINES

World / Asia / Pakistan

Karachi's new AI traffic system issues 2,662 e-challans worth Rs12.5 million in 6 hours

Smart cameras replace manual policing as TRACS ushers in a new era of road discipline

Last updated: October 28, 2025 | 16:12

Stephen N R, *Senior Associate Editor*



Use case:

Traffic Violation Detection: Detect vehicles violating traffic rules like running red lights or wrong-way driving.

Waste Segregation Automation: Detect and classify waste types (plastic, paper, metal) on a conveyor belt for recycling automation.

Types of ANN

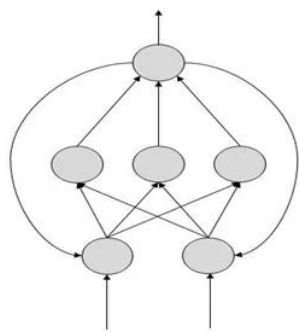
2- FeedBack ANN

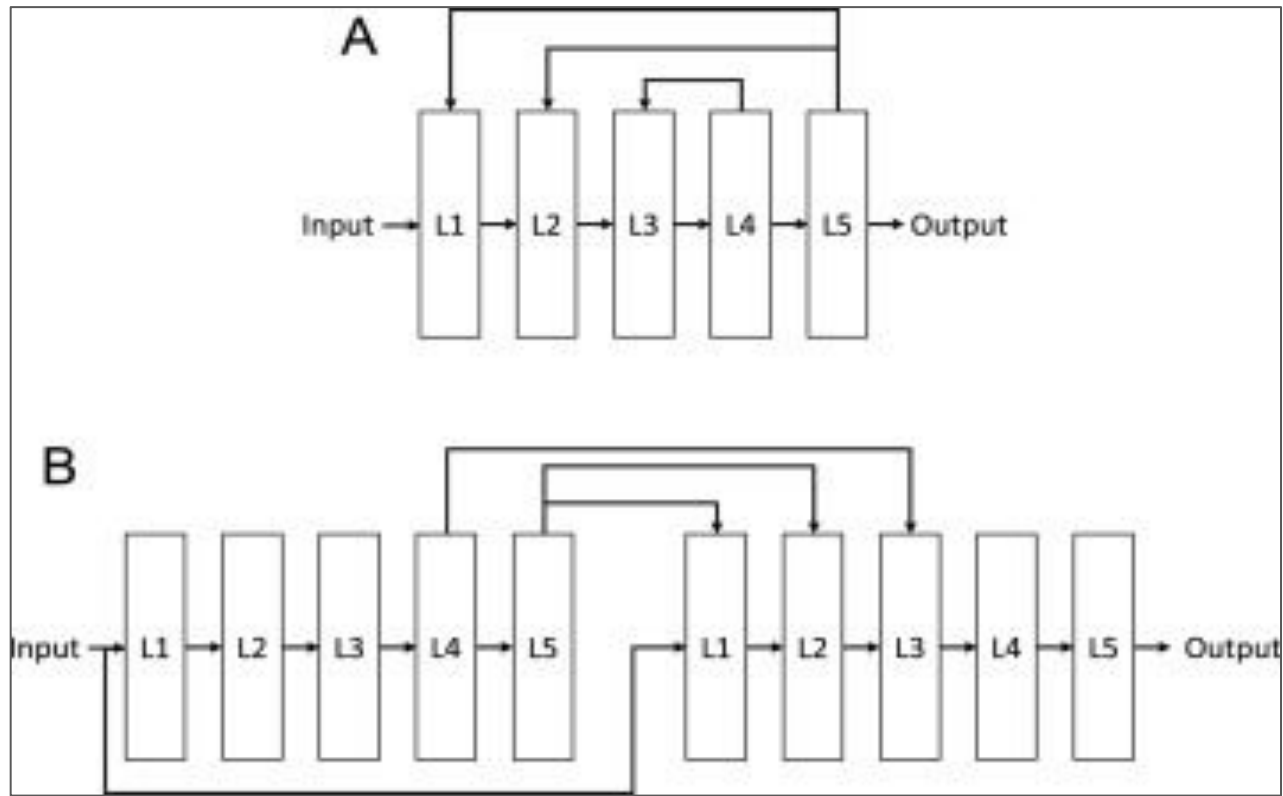
A Feedback ANN (also called a Recurrent Neural Network, or more generally any network with feedback) is a neural network where outputs from neurons are fed back into the network as inputs, either to the same layer or previous layers. This creates loops, unlike feedforward networks.

Example:

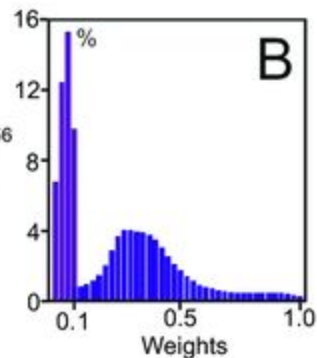
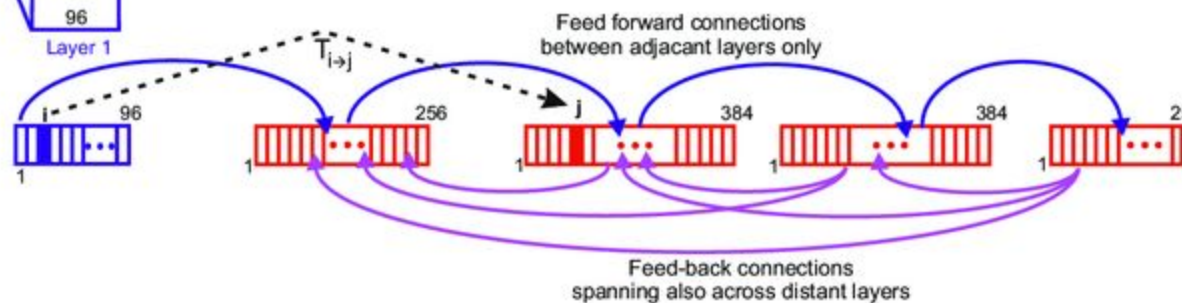
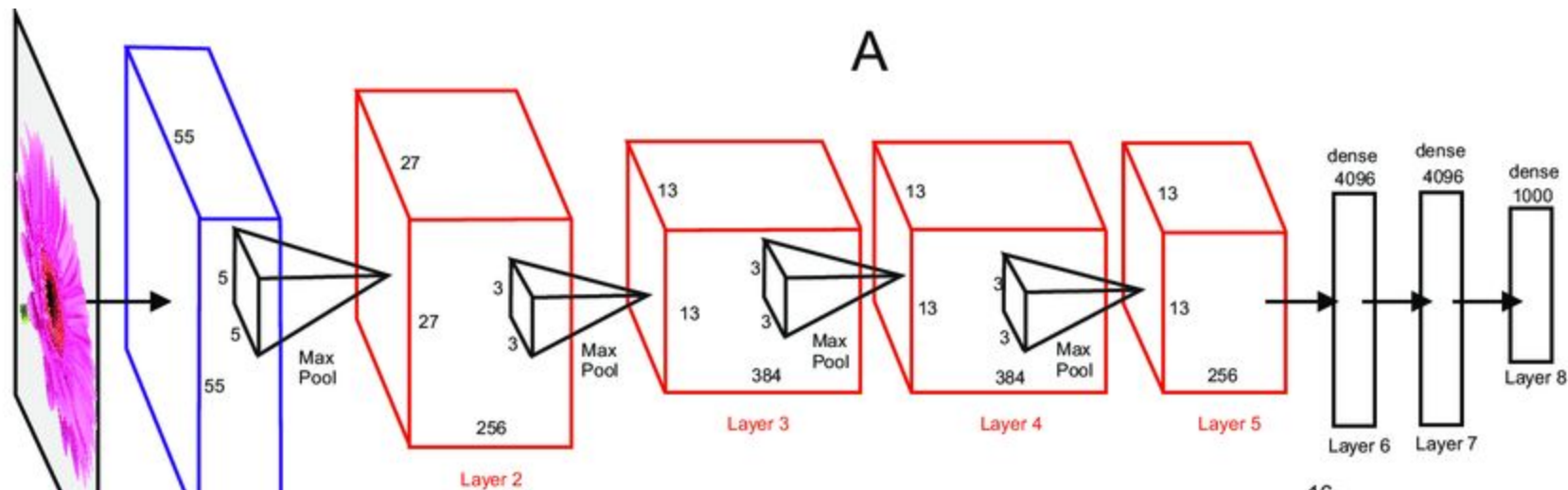
General Recurrent Neural Network

- Suppose you want to predict the next word in a sentence.
 - Feedforward ANN can't remember previous words—each prediction is independent.
 - Feedback ANN (RNN) keeps a “memory” of previous words via feedback loops and can predict based on the sequence.





Schematic representation of the unrolling of the recurrent network. (A) shows the network with feedback connections. (B) shows the unrolled network.



Formula

$$f\left(\sum_{i=1}^3 x_i * w_{1i} + b_1\right)$$

$$f(z) = \textit{Sigmoid}(z) = \frac{1}{1 + e^{-z}}$$

What is Activation Function?

It's just a thing function that you use to get the output of the node. It is also known as Transfer Function.

Types of Activation Function

The Activation Functions can be based on 2 types-

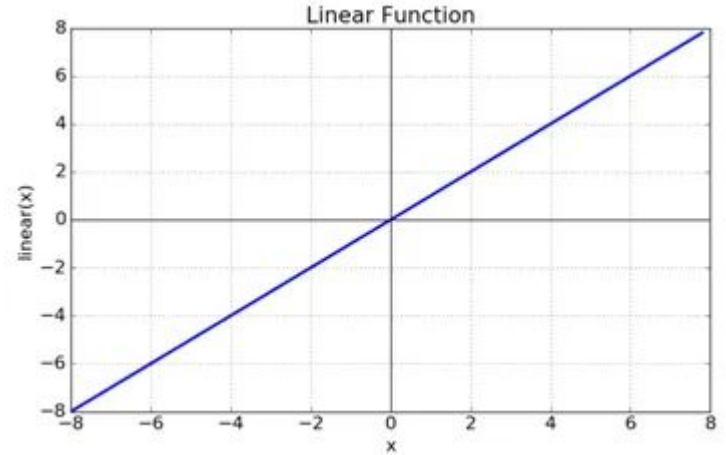
1. Linear Activation Function
2. Non-linear Activation Functions

Linear Activation Function

Equation: $f(x) = x$

Range: (-infinity to infinity)

It doesn't help with the complexity of various parameters of usual data that is fed to the neural networks.

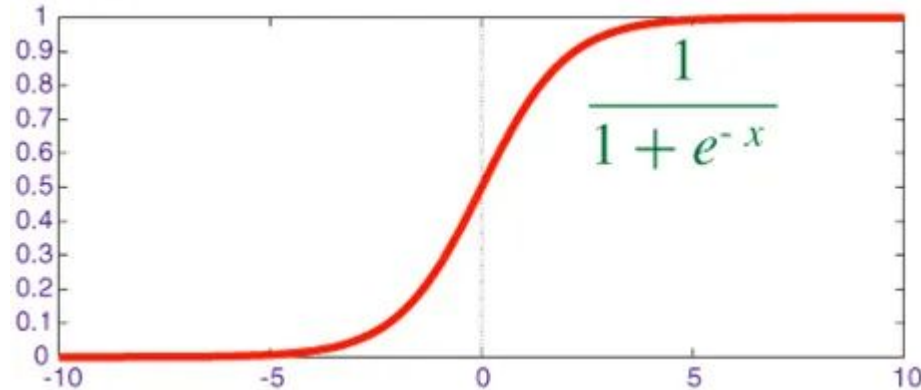


Non Linear Activation Function

Different Types of Activation function in non-Linear

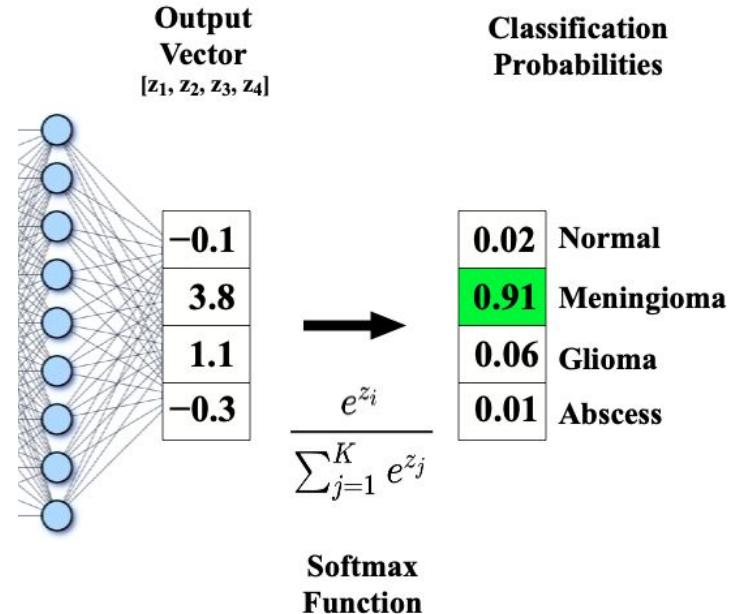
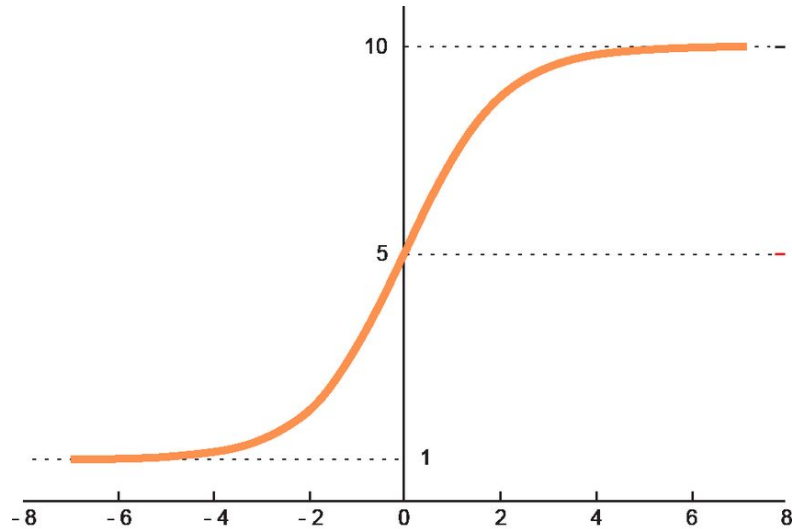
1. Sigmoid Activation Function:

- The Sigmoid Function curve looks like an S-shape.
- The main reason why we use the sigmoid function is that it exists between (0 to 1). Therefore, it is especially used for models where we have to **predict the probability as an output**.
-

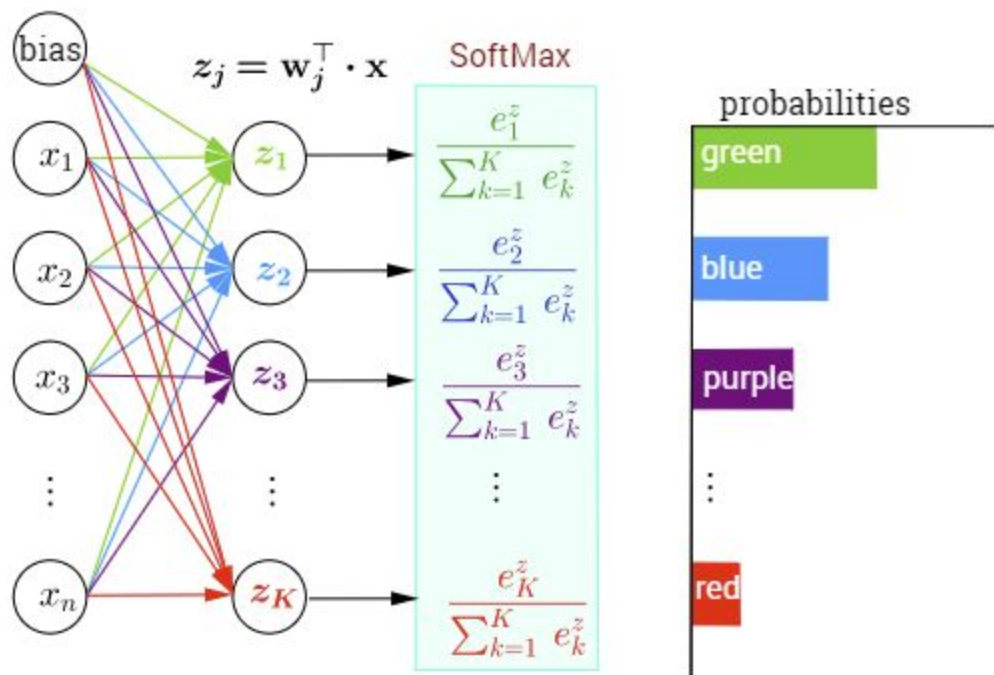


Softmax Activation Function

The softmax function is a more generalized logistic activation function that is used for multiclass classification.

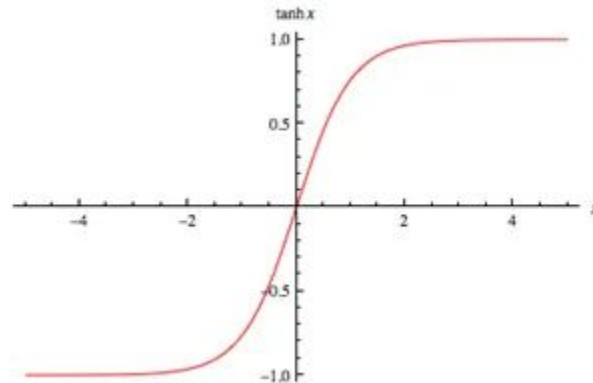


$$\mathbf{z} = \begin{bmatrix} z_1 \\ z_2 \\ z_3 \\ \vdots \\ z_K \end{bmatrix} = \begin{bmatrix} \mathbf{w}_1^\top \\ \mathbf{w}_2^\top \\ \mathbf{w}_3^\top \\ \vdots \\ \mathbf{w}_K^\top \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ \vdots \\ x_n \end{bmatrix}$$



Tanh Activation Function

- $\tanh$ is also like logistic sigmoid but better. The range of the $\tanh$ function is from $(-1$ to $1)$. $\tanh$ is also sigmoidal (s-shaped).
- The $\tanh$ function is mainly used classification between two classes.
- Both $\tanh$ and logistic sigmoid activation functions are used in feed-forward nets.

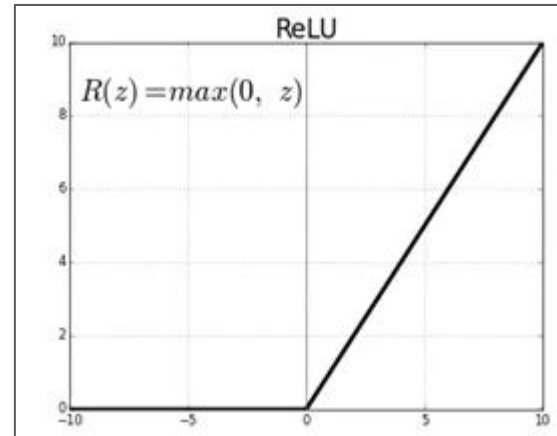


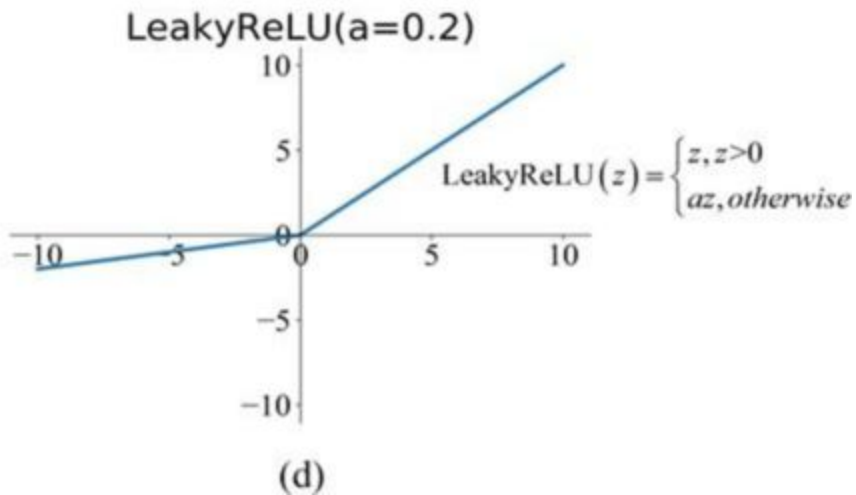
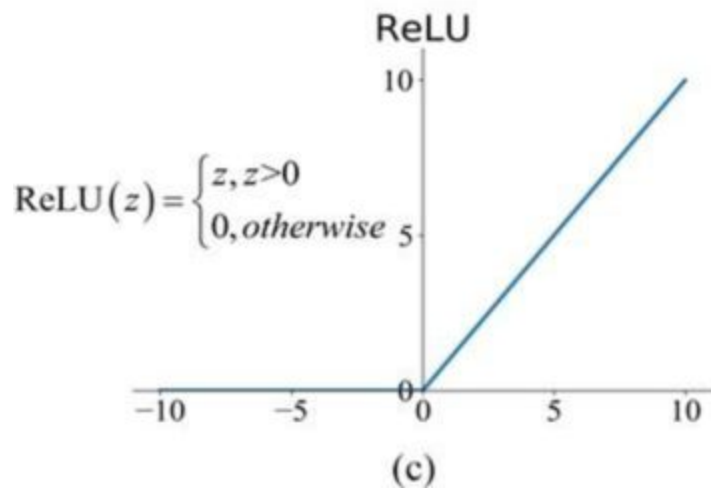
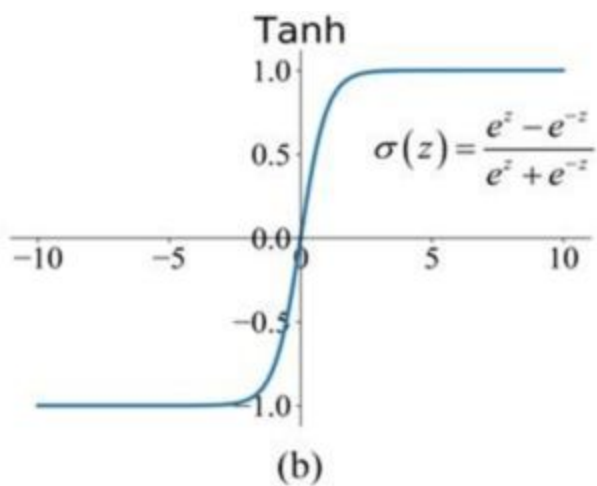
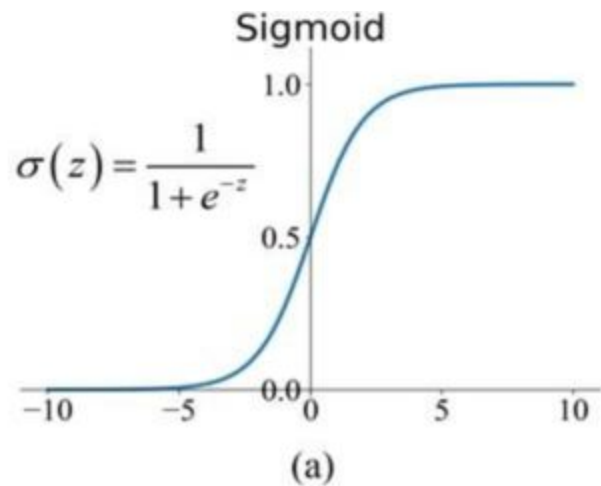
ReLU (Rectified Linear Unit) Activation Function

The ReLU is the most used activation function in the world right now. Since it is used in almost all the convolutional neural networks or deep learning.

As you can see, the ReLU is half rectified (from bottom). **$f(z)$ is zero when z is less than zero and $f(z)$ is equal to z when z is above or equal to zero.**

Range: [0 to infinity]

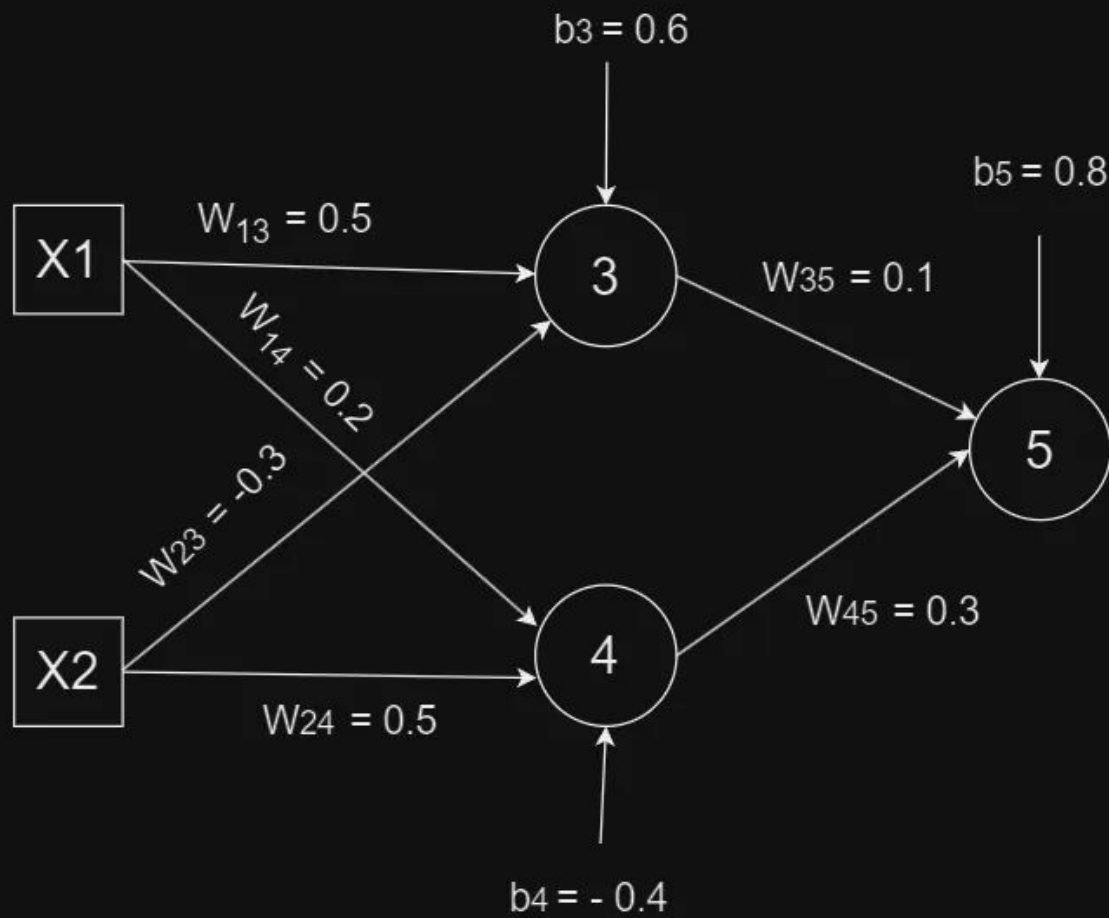




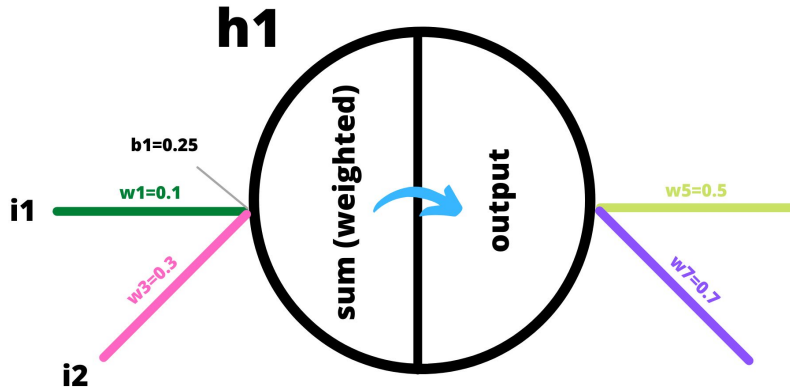
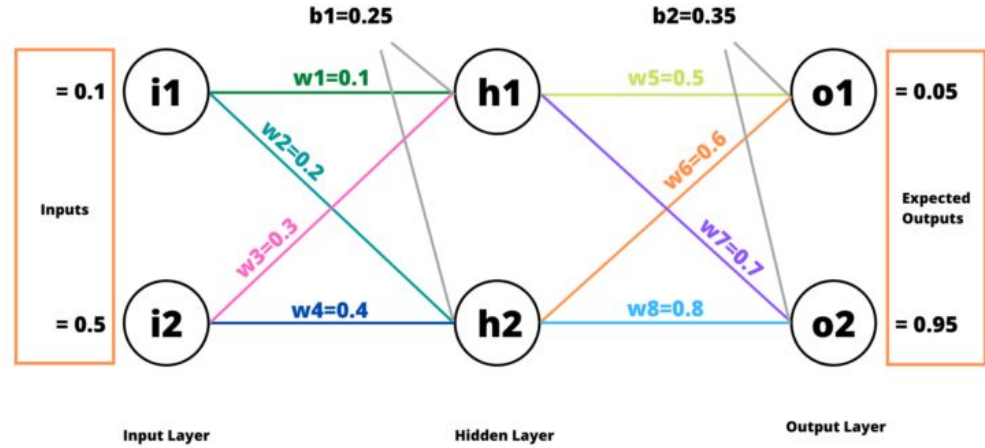
Cost Function

$$L(\mathbf{\hat{y}}^{(j)}, \mathbf{y}^{(j)}) = - \sum_{i=1}^n y_i^{(j)} \ln(\hat{y}_i^{(j)})$$

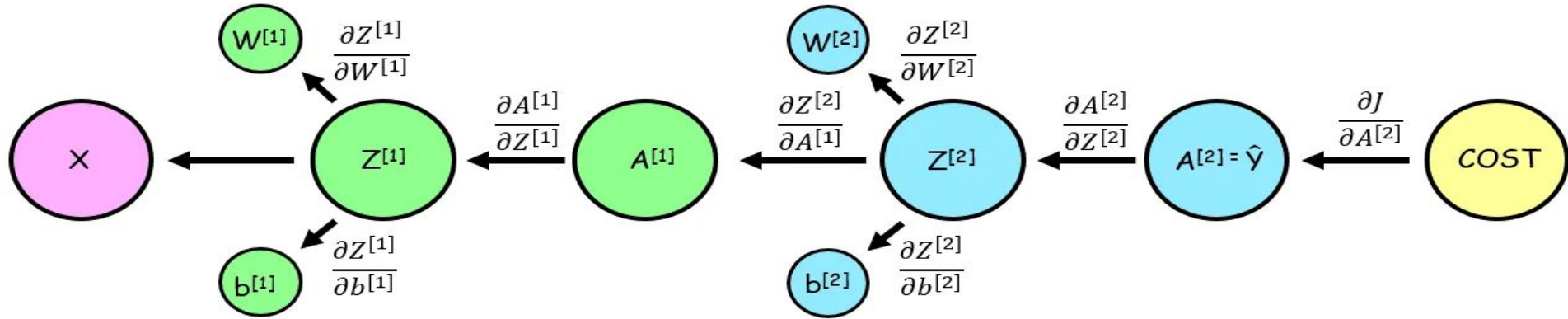
$$J(\mathbf{W}, \mathbf{b}) = -\frac{1}{m} \sum_{j=1}^m \sum_{i=1}^n y_i^{(j)} \ln(\hat{y}_i^{(j)})$$



Forward Pass



How Backpropagation work?



$$\frac{\partial J}{\partial W^{[2]}}$$

,

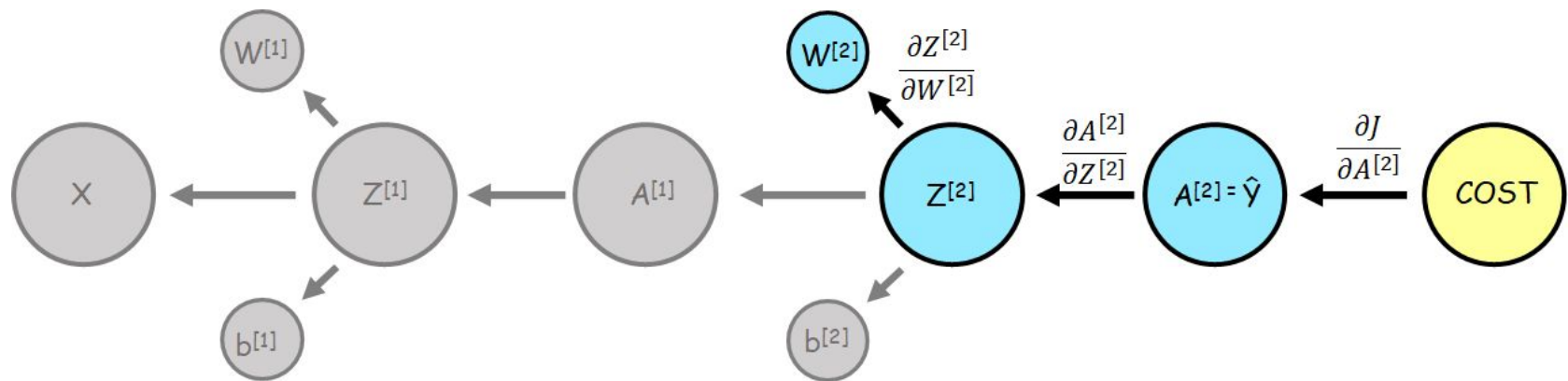
$$\frac{\partial J}{\partial b^{[2]}}$$

,

$$\frac{\partial J}{\partial W^{[1]}}$$

,

$$\frac{\partial J}{\partial b^{[1]}}$$



$$\frac{\partial J}{\partial W^{[2]}} = \frac{\partial J}{\partial A^{[2]}} \frac{\partial A^{[2]}}{\partial Z^{[2]}} \frac{\partial Z^{[2]}}{\partial W^{[2]}}$$

$$dj/dA^{[2]}$$

Original function:

$$J = \frac{1}{2} (A^{[2]} - Y)^2$$

Partial derivative:

$$| \quad \frac{\partial J}{\partial A^{[2]}} = A^{[2]} - Y$$

$$dA^{[2]}/dZ^{[2]}$$

Original function:

$$A^{[2]} = \sigma(Z^{[2]}) = \frac{1}{1 + e^{-Z^{[2]}}}$$

Partial derivative:

$$| \quad \frac{\partial A^{[2]}}{\partial Z^{[2]}} = A^{[2]}(1 - A^{[2]})$$

$$dZ^{[2]}/dW^{[2]}$$

Original function:

$$Z^{[2]} = W^{[2]}A^{[1]} + b^{[2]}$$

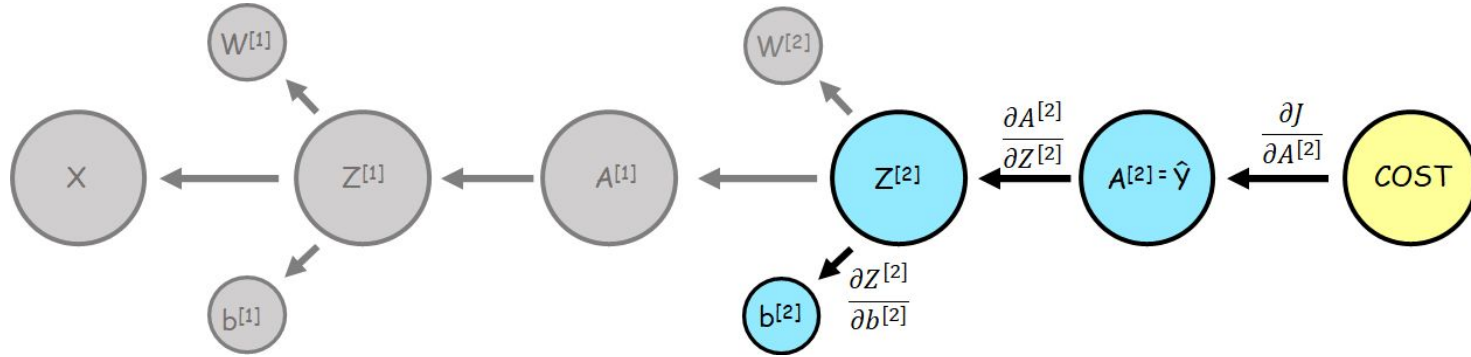
Partial derivative:

$$| \quad \frac{\partial Z^{[2]}}{\partial W^{[2]}} = A^{[1]}$$

$$\frac{\partial J}{\partial W^{[2]}} = \frac{\partial J}{\partial A^{[2]}} \frac{\partial A^{[2]}}{\partial Z^{[2]}} \frac{\partial Z^{[2]}}{\partial W^{[2]}} = (A^{[2]} - Y) \cdot A^{[2]}(1 - A^{[2]}) \cdot A^{[1]}$$

Equation 11. Gradient of the cost J with respect to the weights of layer two ($W^{[2]}$)

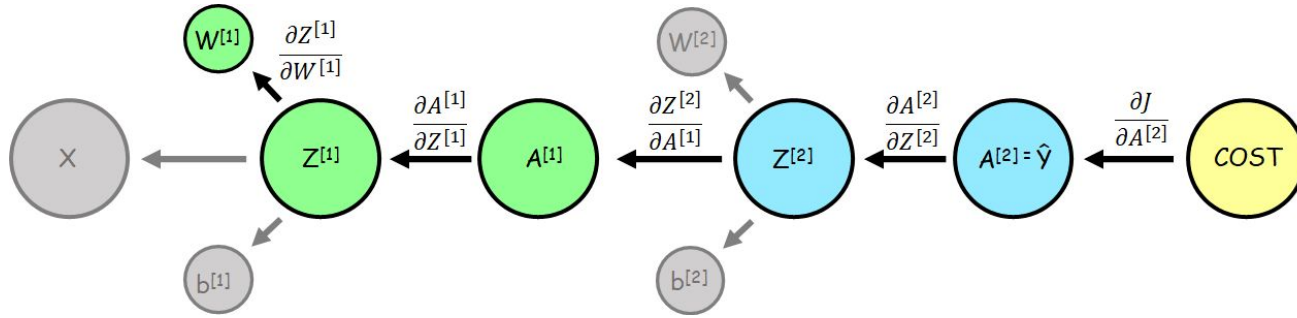
Similarly for $b^{[2]}$



$$\frac{\partial J}{\partial b^{[2]}} = \frac{\partial J}{\partial A^{[2]}} \frac{\partial A^{[2]}}{\partial Z^{[2]}} \frac{\partial Z^{[2]}}{\partial b^{[2]}} = (A^{[2]} - Y) \cdot A^{[2]}(1 - A^{[2]}) \cdot 1$$

Equation 14. The gradient of the cost J with respect to the weights of layer two $W^{[2]}$

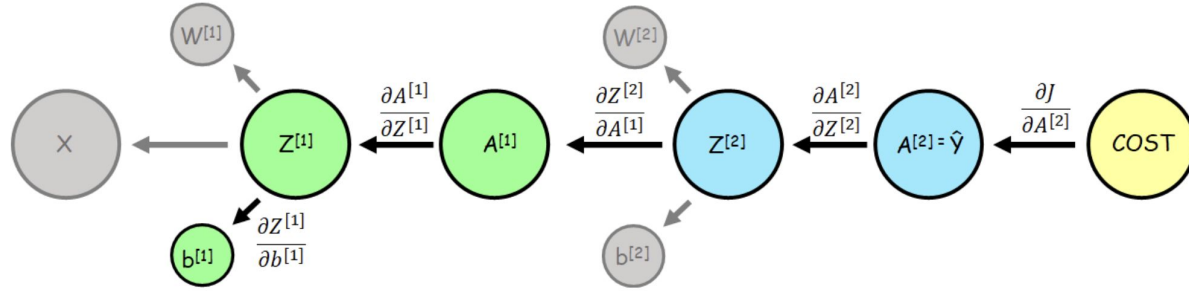
For the $W^{[1]}$



$$\frac{\partial J}{\partial W^{[1]}} = \underbrace{\frac{\partial J}{\partial A^{[2]}} \frac{\partial A^{[2]}}{\partial Z^{[2]}} \frac{\partial Z^{[2]}}{\partial A^{[1]}} \frac{\partial A^{[1]}}{\partial Z^{[1]}}}_{\text{Previously calculated}} \frac{\partial Z^{[1]}}{\partial W^{[1]}} = (A^{[2]} - Y) \cdot A^{[2]}(1 - A^{[2]}) \cdot W^{[2]} \cdot A^{[1]}(1 - A^{[1]}) \cdot X$$

Equation 16. The gradient of the cost J with respect to the weights of the first layer $W^{[1]}$

For $b^{[1]}$



$$\frac{\partial J}{\partial b^{[1]}} = \underbrace{\frac{\partial J}{\partial A^{[2]}} \frac{\partial A^{[2]}}{\partial Z^{[2]}}}_{\text{Previously calculated}} \frac{\partial Z^{[2]}}{\partial A^{[1]}} \frac{\partial A^{[1]}}{\partial Z^{[1]}} \frac{\partial Z^{[1]}}{\partial b^{[1]}} = (A^{[2]} - Y) \cdot A^{[2]}(1 - A^{[2]}) \cdot W^{[2]} \cdot A^{[1]}(1 - A^{[1]}) \cdot 1$$

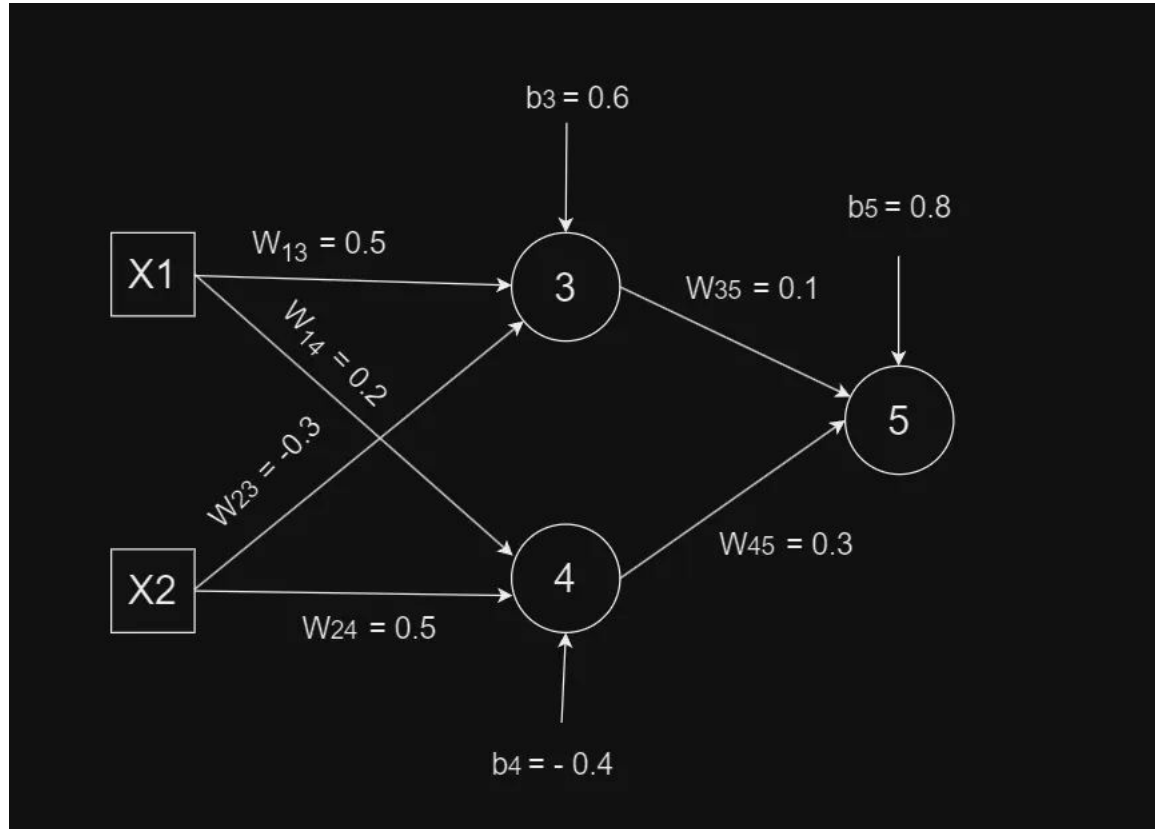
Equation 17. The gradient of the cost J with respect to the bias of the first layer $b^{[1]}$

Updated weights and biases

$$\begin{array}{lcl} W_{new}^{[1]} = W_{old}^{[1]} - \alpha \frac{dJ}{dW^{[1]}} & | & b_{new}^{[1]} = b_{old}^{[1]} - \alpha \frac{dJ}{db^{[1]}} \\ W_{new}^{[2]} = W_{old}^{[2]} - \alpha \frac{dJ}{dW^{[2]}} & | & b_{new}^{[2]} = b_{old}^{[2]} - \alpha \frac{dJ}{db^{[2]}} \end{array}$$

W13	W14	W23	W24	W35	W45	b3	b4	b5
0.5	0.2	-0.3	0.5	0.1	0.3	0.6	-0.4	0.8

How
Forward
Propagati
on works?



1st sample data

X1	X2	Output(τ)
1	1	0

W13	W14	W23	W24	W35	W45	b3	b4	b5
0.5	0.2	-0.3	0.5	0.1	0.3	0.6	-0.4	0.8

c. Learning rate (η) = 0.5

d. Activation function = Sigmoid function

$$\text{(i.e. } \sigma = \frac{1}{1 + e^{-x}} \text{)}$$

Task

Iteration 2

Input and output values:

x1	x2	Output(τ)
0	1	1

Input and output for iteration 2

Weight(W) and bias(b) values:

Take the updated weights and bias we had got in the previous step.

Answers

W13	W14	W23	W24	W35	W45	b3	b4	b5
0.4848	0.1948	-0.315	0.4967	0.0678	0.2747	0.5852	-0.403	0.7587

Numerical

One numerical is done during class.

Practical Implementation of Artificial Neural Network

<https://colab.research.google.com/drive/11Az2oliNAaCdT-iYZTsxw1dYchXn3MHw?usp=sharing>

Sequential Models

A sequential model is a type of machine learning model that processes data in order, where **the current output or state depends on previous inputs.**

These models are especially useful for time-series data, language, or sequences where the order matters.

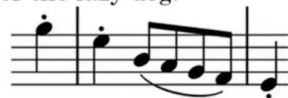
Examples of sequence data

Speech recognition



“The quick brown fox jumped over the lazy dog.”

Music generation



Sentiment classification

“There is nothing to like in this movie.”



DNA sequence analysis

AGCCCCTGTGAGGAACTAG



AG**CC**CCTGTGAGG**AA**CTAG

Machine translation

Voulez-vous chanter avec moi?



Do you want to sing with me?

Video activity recognition



Running

Name entity recognition

Yesterday, Harry Potter met Hermione Granger.



Yesterday, **Harry Potter** met **Hermione Granger**.

Andrew Ng

Drawbacks to non-sequential models

Earlier, we used methods like Bag-of-Words (BoW) to convert text into numbers so that a computer could process it. Although this method works for basic text tasks, it has several limitations:

1. **Out-of-Vocabulary (OOV) Words**
If a new word appears that wasn't seen during training, the model doesn't know what it means — it just ignores it.
2. **Sparse and Inefficient Representations**
BoW creates large matrices full of zeros (sparse data), which is inefficient to store and process.
3. **No Understanding of Word Relationships**
BoW treats all words as separate items — it doesn't know that “happy” and “joyful” have similar meanings.
4. **No Word Order Information**
Word order is completely lost.
For example:
 - “They eat bread”
 - “Bread eat they”These two sentences mean very different things, but BoW gives them the same representation.
5. **Static Embeddings Don't Capture Sequence**
Even when we use word embeddings (like Word2Vec or GloVe), they only give a fixed meaning to each word, not depending on context or position in a sentence.

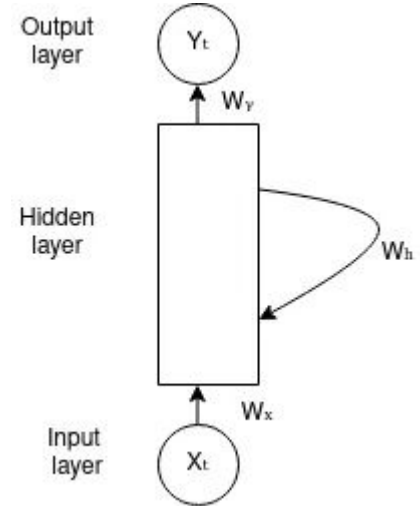
Why FNNs Are Not Suitable for Sequential Data

Key Reasons

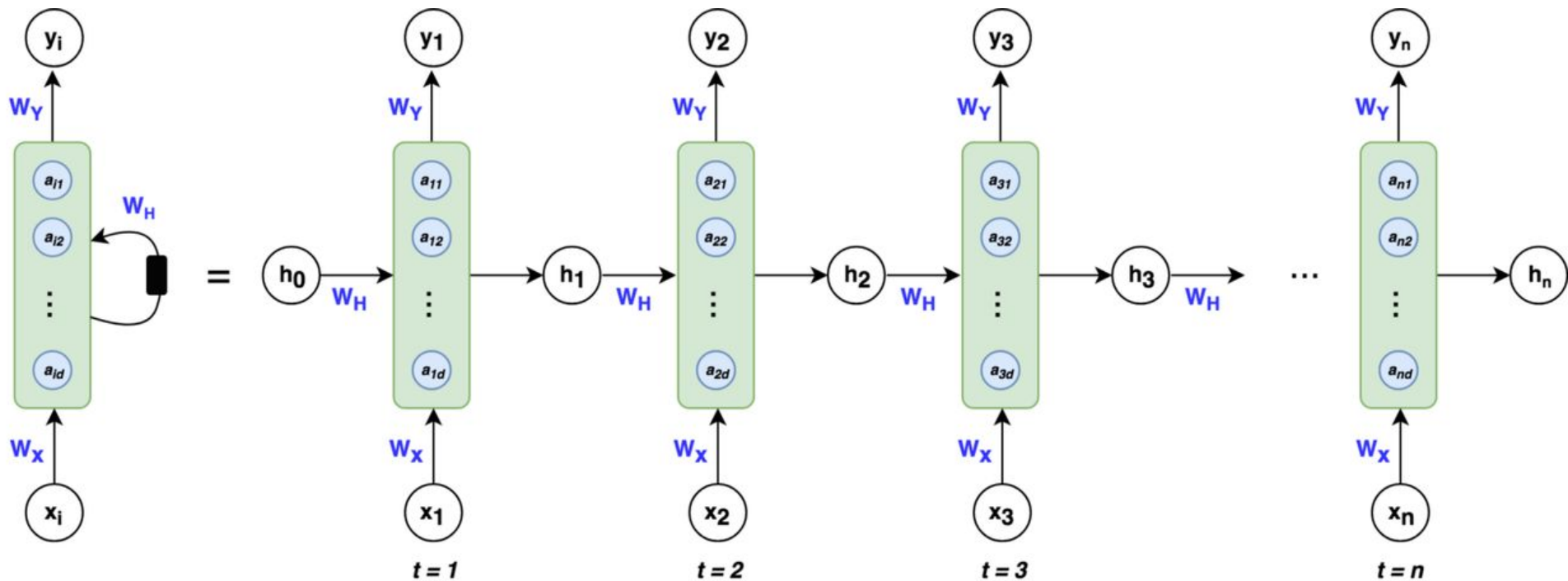
1. **No memory of previous inputs**
→ FNN treats each input separately.
Example: Cannot understand a sentence like “He went to the bank to deposit money” because it forgets the earlier context.
2. **Cannot handle variable-length sequences**
→ Requires fixed-size vectors.
Example: Sentences of 5 words vs. 20 words cannot be processed naturally.
3. **No understanding of order or time**
→ Word position or time-step meaning is lost.
Example: “Man bites dog” vs. “Dog bites man” both look same to an FNN.
4. **No parameter sharing across time**
→ Each position has different weights → inefficient for patterns that repeat.
Example: Daily weather patterns or repeated phrases in speech.
5. **Cannot capture long-term dependencies**
→ No hidden state or memory flow.
Example: Predicting the next word in “I like to drink ...” needs earlier words.

RNN

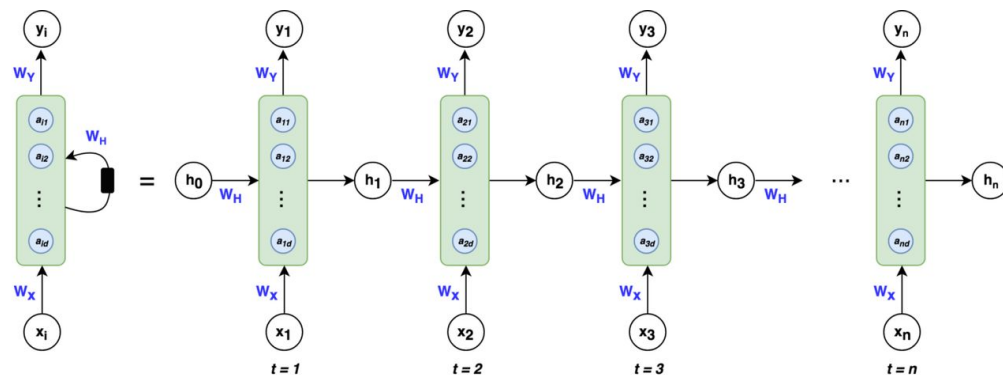
RNN overcome these limitations by introducing a recurrent connection that allow information to flow from one time-step to the next. This recurrent connection enables RNNs to maintain internal memory, where the output of each step is fed back as an input to the next step, allowing the network to capture the information from previous steps and utilize it in the current step, enabling model to learn temporal dependencies and handle input of variable length.



Recurrence



Recurrence



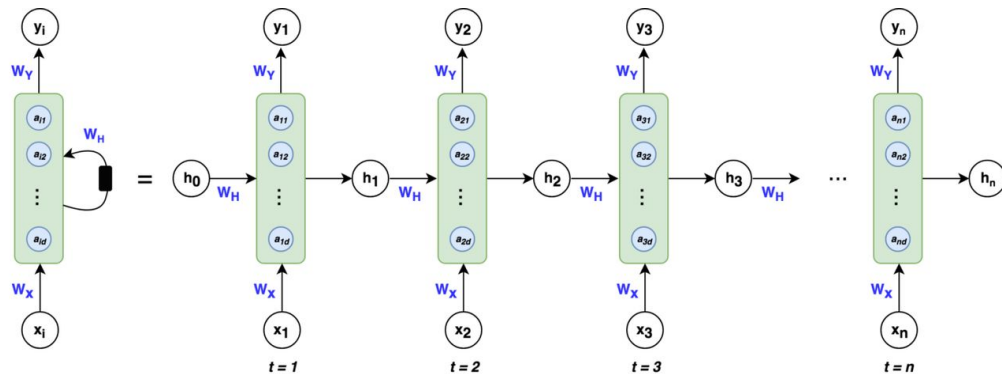
- The recurrence in an RNN refers to the repeating design of the model.
- An input word vector (x_t) is passed through a fully connected layer to generate some activations (a_t), which can be used to make predictions (y_t) at that time step.
- But these activations are also passed to the next time step acting as a form of memory, which are combined to make the next prediction (y_{t+1}).
- This design is repeated at each time step, so there can be any number of units in the sequence.

Recurrence

There are 3 weights matrices, which are used repeatedly used in each cell.

W_x are the weights for the input, W_y are the weights for the output and W_h are the weights for the memory. They are combined using the following equations:

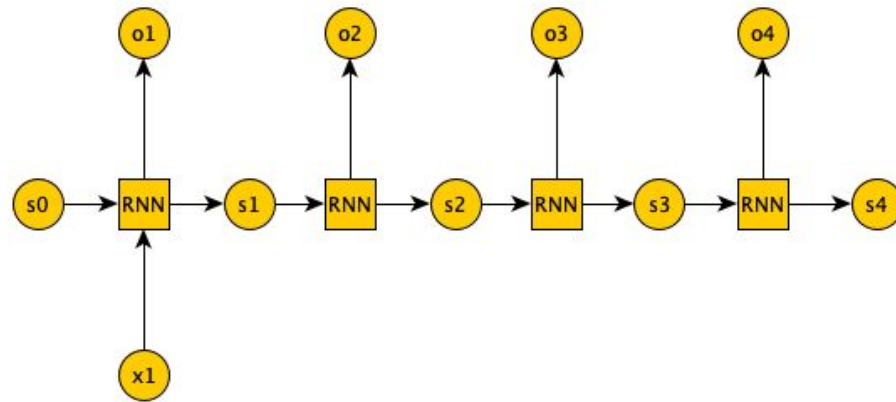
$$h_t = f(W_x x_t + W_h h_{t-1} + b), \quad y_t = g(W_y a_t + c)$$



RNN

- RNN models sequential data (temporal/ordinal)
- In RNN, training example is a sequence, which is presented to RNN one at a time
 - E.g., sequence of English words is passed to RNN, one at a time
 - And, RNN generates a sequence of Persian words, one at a time
- In RNN, output of network at time t is used as input at time $t+1$
- RNN applied to image description, machine translation, sentiment analysis, etc.

Dog on the couch



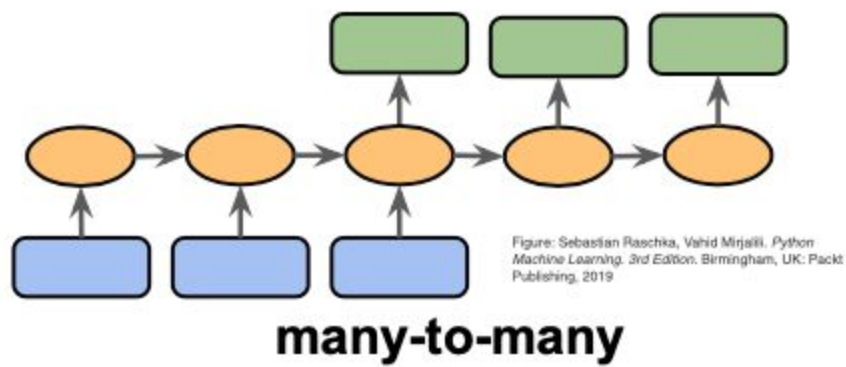
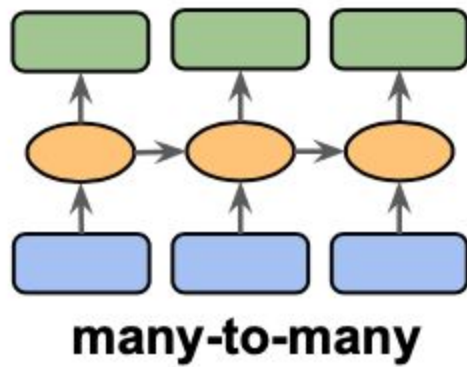
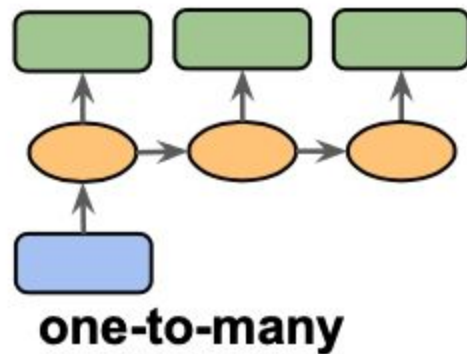
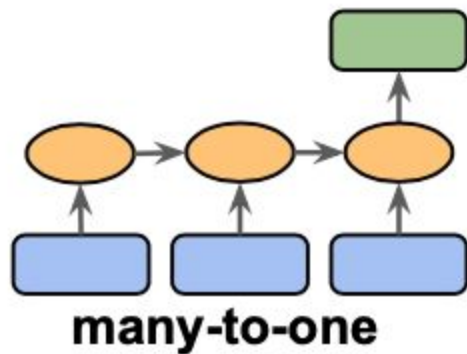
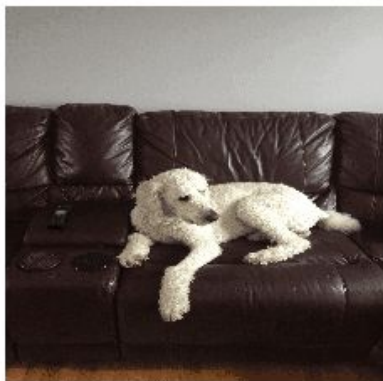
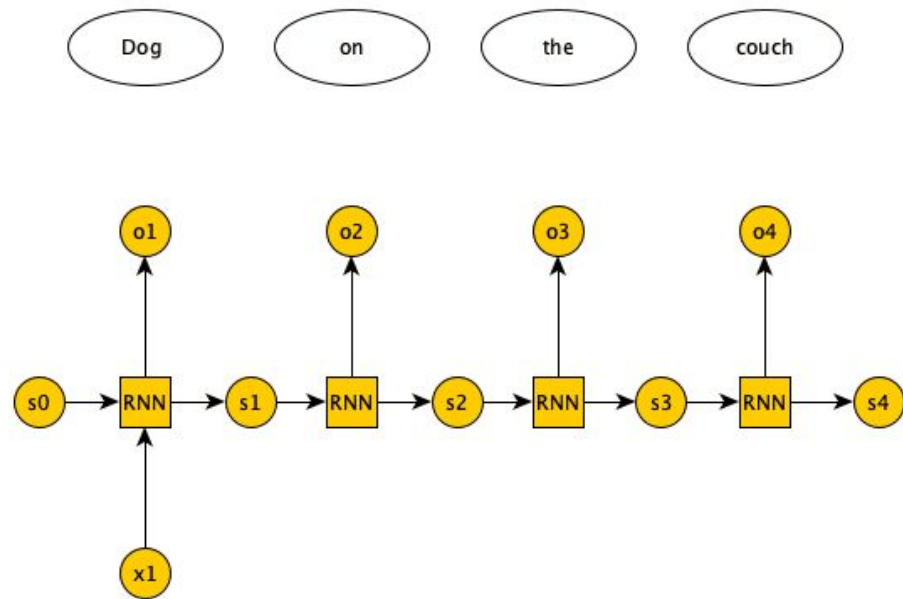
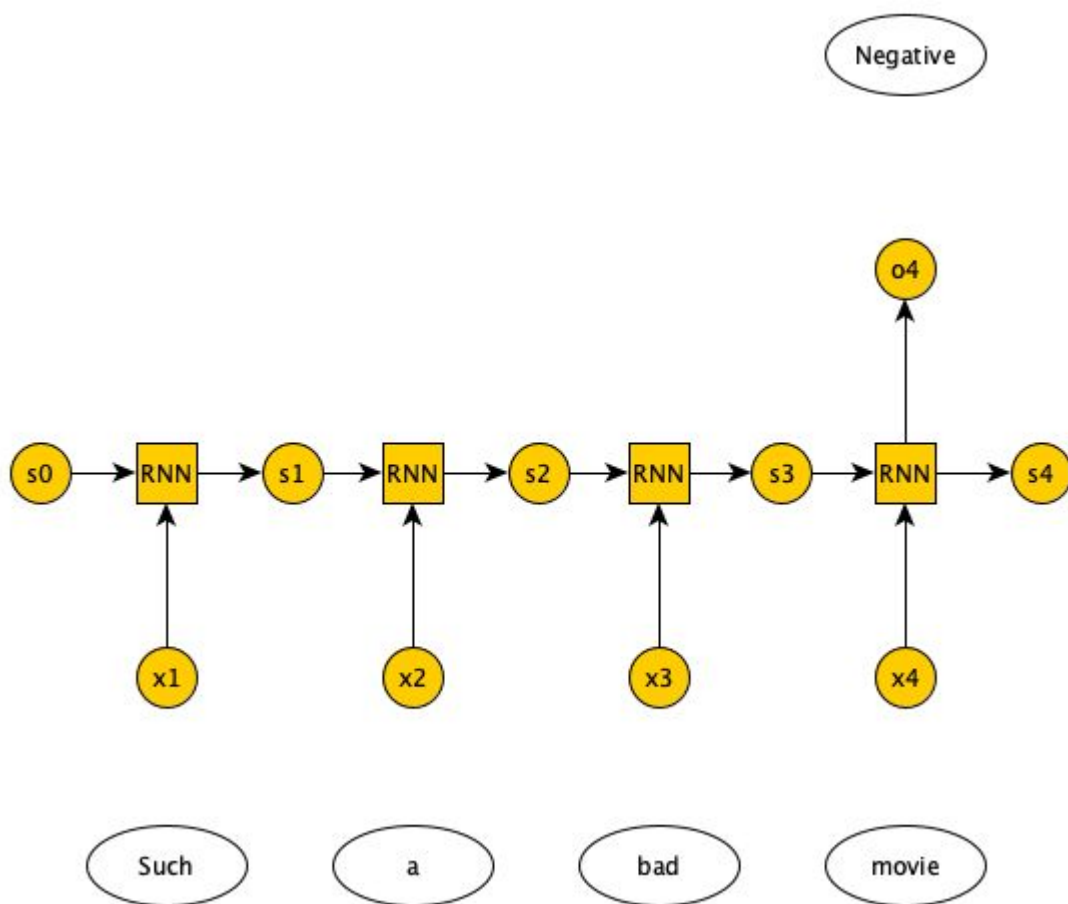


Figure: Sebastian Raschka, Vahid Mirjalili. Python Machine Learning, 3rd Edition. Birmingham, UK: Packt Publishing, 2019

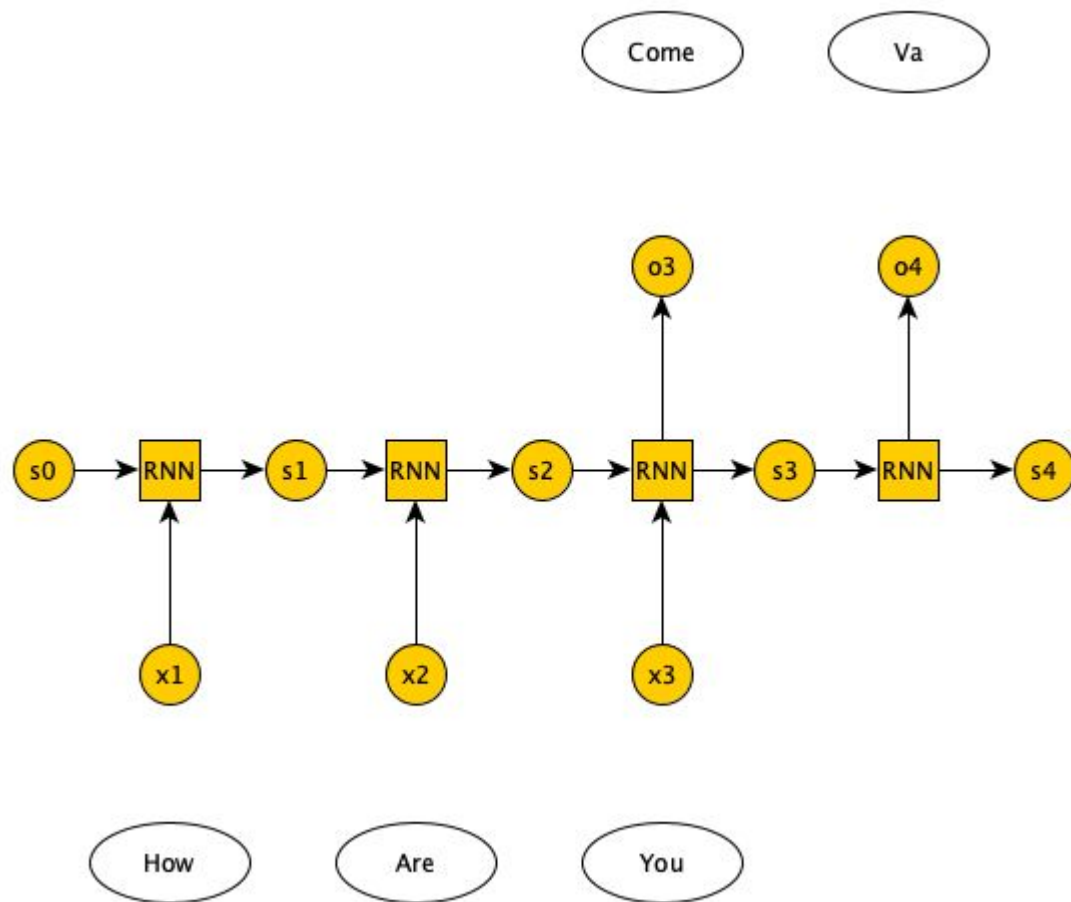
One to Many RNN



Many to One RNN

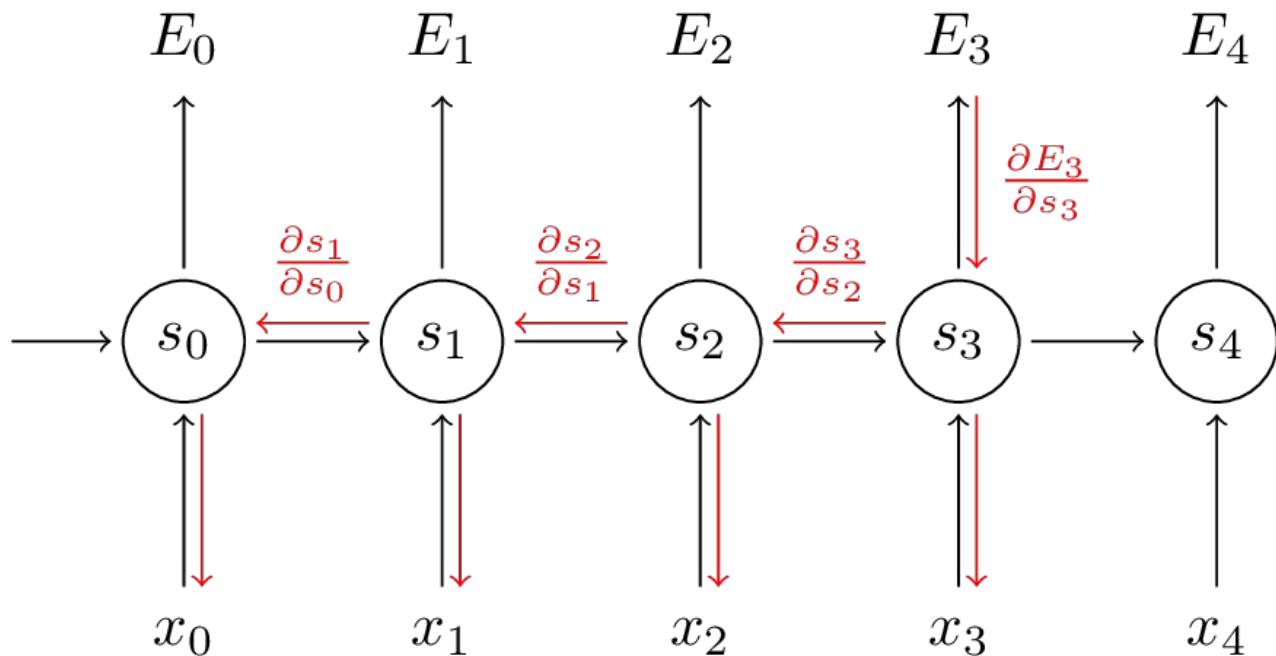


Many to Many RNN



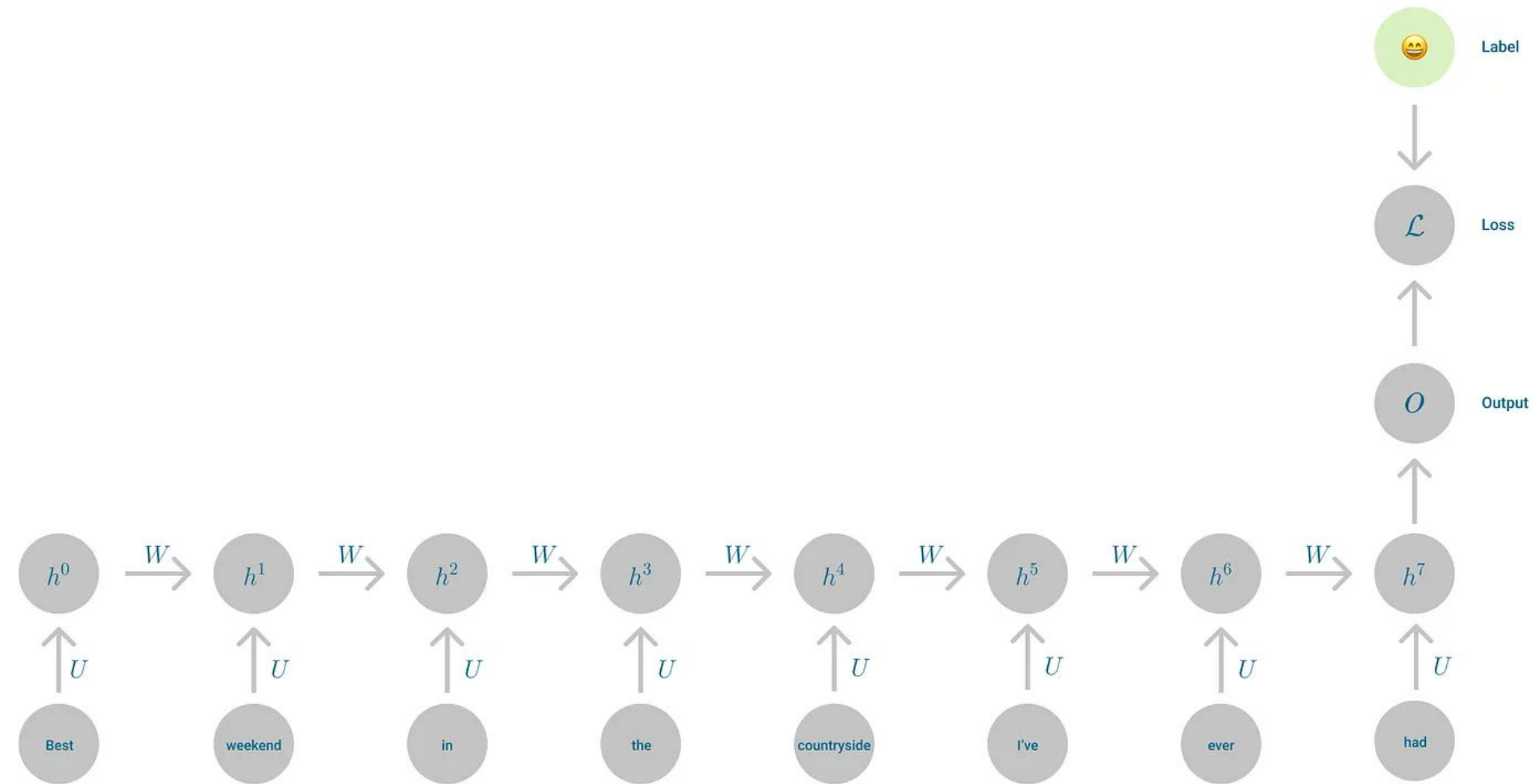
Training RNNs

Training an RNN is **trickier** than a standard feed-forward neural network because of the time aspect. The algorithm used is called **Backpropagation Through Time** (BPTT). The main idea is that when we compute gradients, we need to **reverse all the arrows** in the computation graph and multiply them using the chain rule.



Mean Square Error

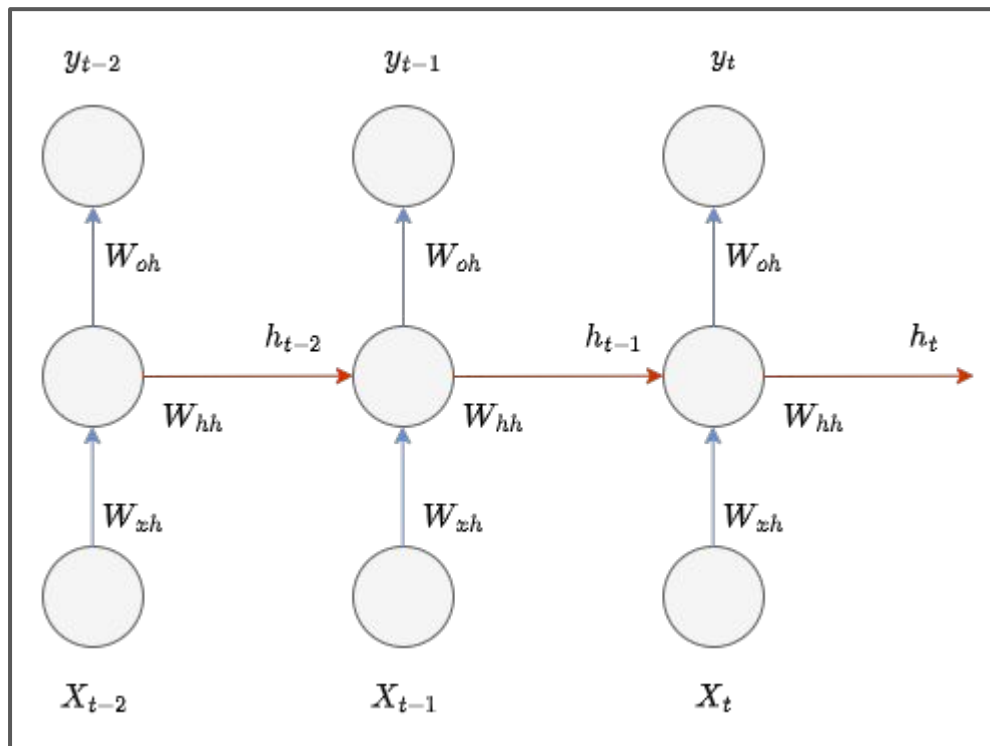
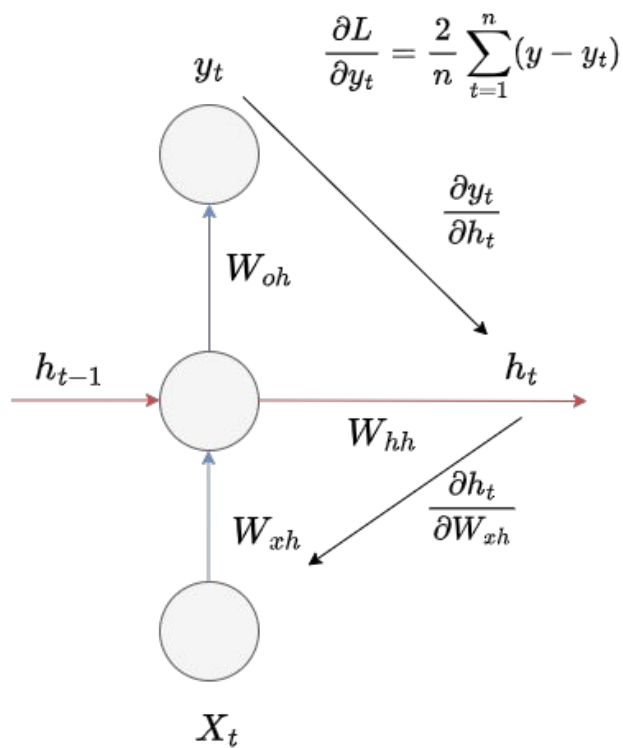
$$L(y, \hat{y}) = (1/t) * \sum (y_i - \hat{y}_i)^2 \longrightarrow \text{MSE Loss}$$



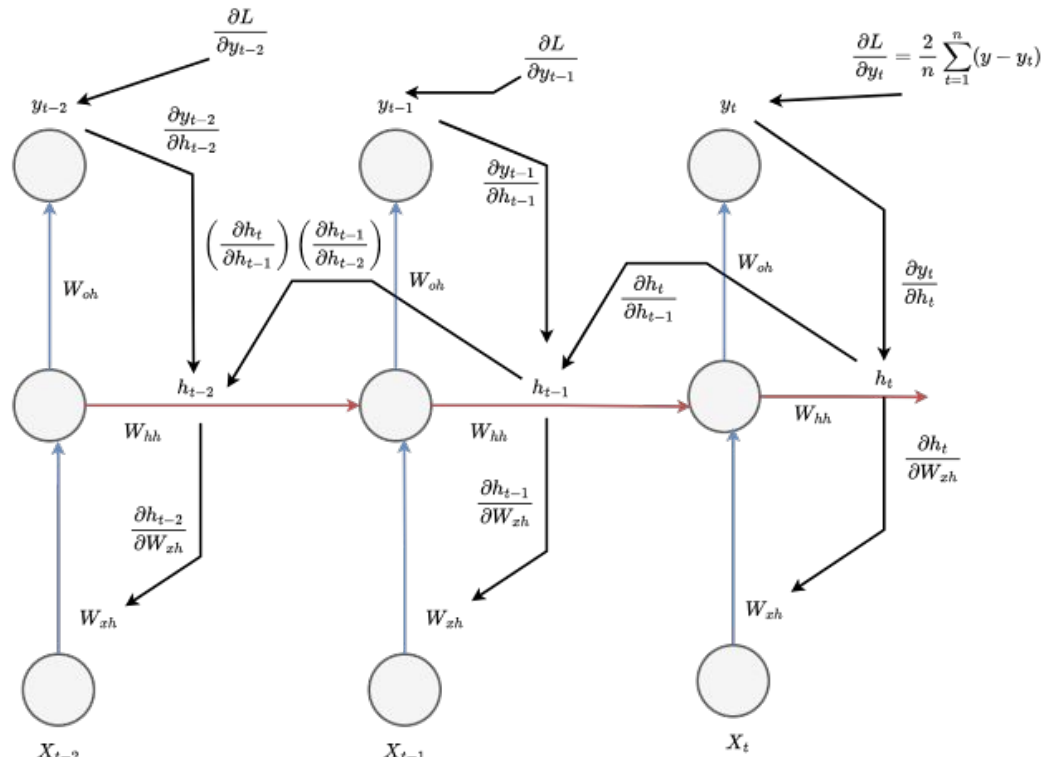
Backpropagation Through Time (BPTT)

Backpropagation Through Time is a special variant of backpropagation used to train RNNs, where the error is propagated backward through time until the initial time step $t=1$. Backpropagation involves two key steps: forward pass and backward pass.

BPTT



BPTT



$$\frac{\partial L}{\partial W_{xh}} = \underbrace{\frac{\partial L}{\partial y_t} \frac{\partial y_t}{\partial h_t} \frac{\partial h_t}{\partial W_{xh}}}_{i=0} + \underbrace{\frac{\partial L}{\partial y_{t-1}} \frac{\partial y_{t-1}}{\partial h_{t-1}} \left(\frac{\partial h_t}{\partial h_{t-1}} \right) \frac{\partial h_{t-1}}{\partial W_{xh}}}_{i=1} + \underbrace{\frac{\partial L}{\partial y_{t-2}} \frac{\partial y_{t-2}}{\partial h_{t-2}} \left(\frac{\partial h_t}{\partial h_{t-1}} \right) \left(\frac{\partial h_{t-1}}{\partial h_{t-2}} \right) \frac{\partial h_{t-2}}{\partial W_{xh}}}_{i=2}$$

BPTT

$$\frac{\partial L}{\partial W_{xh}} = \sum_{i=0}^t \left[\underbrace{\frac{\partial L}{\partial y_{t-i}} \frac{\partial y_{t-i}}{\partial h_{t-i}}}_{R_{t-i}} \left(\prod_{j=(t-i+1)}^t \frac{\partial h_j}{\partial h_{j-1}} \right) \underbrace{\frac{\partial h_{t-i}}{\partial W_{xh}}}_{S_{t-i}} \right]$$

$$\frac{\partial L}{\partial W_{hh}} = \sum_{i=0}^t \left[\underbrace{\frac{\partial L}{\partial y_{t-i}} \frac{\partial y_{t-i}}{\partial h_{t-i}}}_{R_{t-i}} \left(\prod_{j=(t-i+1)}^t \frac{\partial h_j}{\partial h_{j-1}} \right) \underbrace{\frac{\partial h_{t-i}}{\partial W_{hh}}}_{T_{t-i}} \right]$$

$$\frac{\partial L}{\partial W_{oh}} = \sum_{i=0}^t \frac{\partial L}{\partial y_{t-i}} \frac{\partial y_{t-i}}{\partial W_{oh}}$$

Vanishing and/or exploding gradients

Vanishing Gradient Problem

Vanishing gradients occur when gradients become extremely small during backpropagation, causing early layers to learn very slowly or stop learning. During backpropagation the gradient of the loss L with respect to a weight w_i in layer i is calculated using the chain rule:

$$\frac{\partial L}{\partial w_i} = \frac{\partial L}{\partial a_n} \cdot \frac{\partial a_n}{\partial a_{n-1}} \cdot \frac{\partial a_{n-1}}{\partial a_{n-2}} \cdots \frac{\partial a_1}{\partial w_i}$$

When activation functions like Sigmoid or Tanh are used, their derivatives are less than 1. Repeated multiplication through layers causes the gradient to vanish as it moves backwards, making the lower layers unable to learn.

Vanishing and/or exploding gradients

Exploding Gradient Problem

Exploding gradients occur when gradients grow too large during backpropagation, leading to unstable weight updates and divergence in loss. When derivatives or weights are greater than 1, their repeated multiplication across layers leads to exponential growth.

$$\prod_{i=1}^n \frac{\partial a_i}{\partial a_{i-1}} \longrightarrow \infty$$

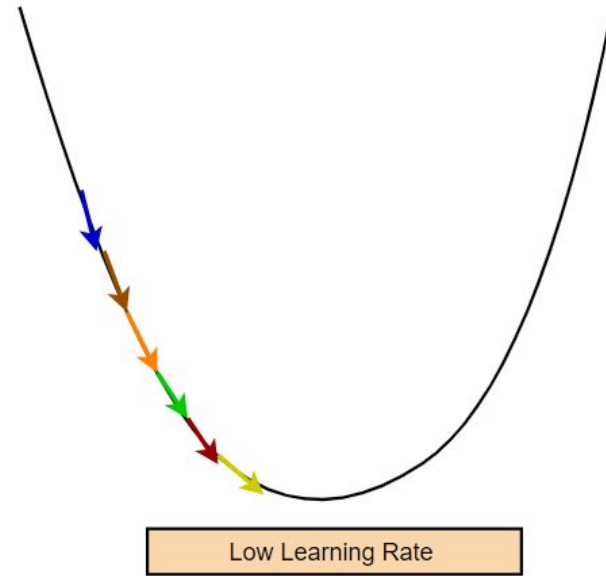
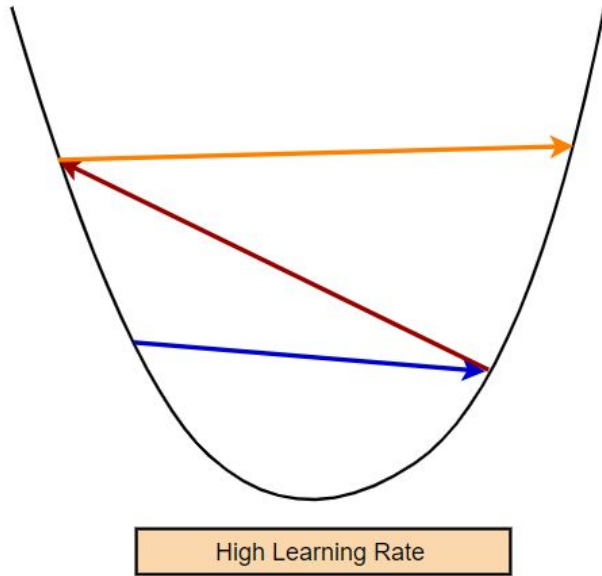
The gradient update rule in gradient descent is:

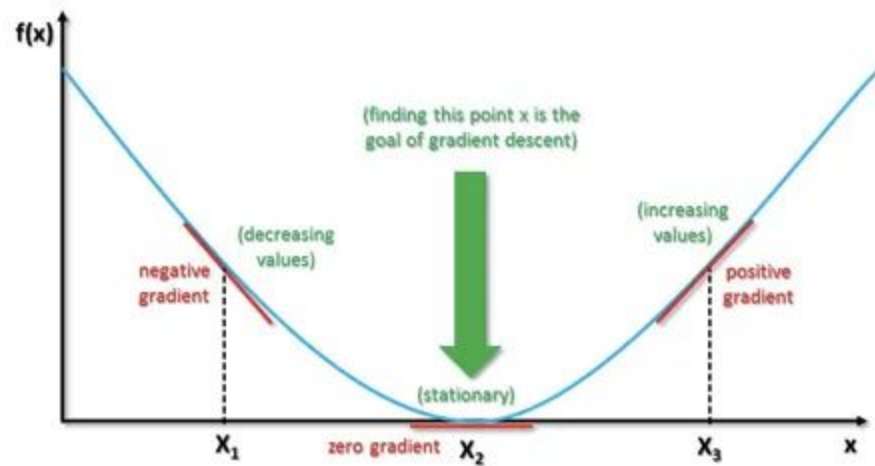
$$w_{t+1} = w_t - \eta \cdot \frac{\partial L}{\partial w_t}$$

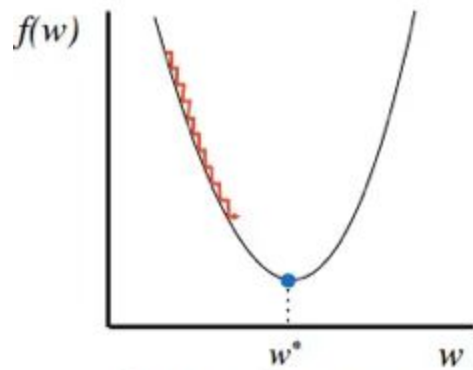
If $\partial L / \partial w_t$ is too large
weight updates
become massive
causing the model loss
to oscillate or diverge.

Why do the Gradients Vanish or Explode

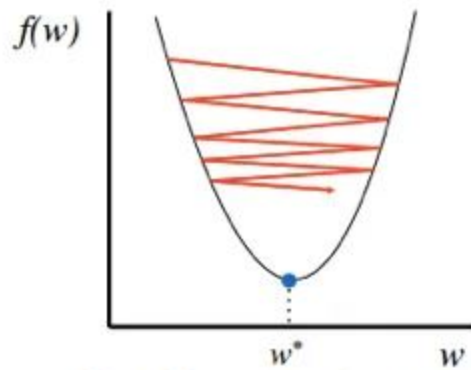
Learning Rate: High learning rate or unscaled inputs can make gradients explode.







Too small: converge
very slowly



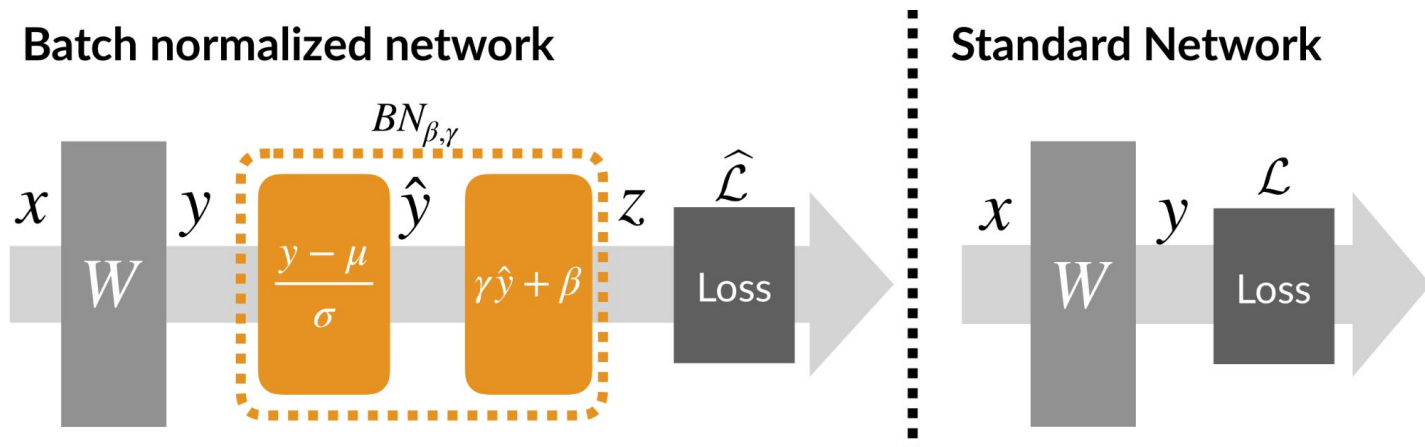
Too big: overshoot and
even diverge

Why do the Gradients Vanish or Explode

- **Activation Functions:** Sigmoid or Tanh have small derivatives that shrink gradients.
- **Weight Initialization:** Too small or too large weights cause vanishing or exploding gradients.
- **Deep Networks:** Many layers multiply gradients repeatedly leading to instability.

Techniques to Fix Vanishing and Exploding Gradients

1. Proper Weight Initialization
 - a. Xavier Initialization
 - b. Kaiming Initializations
2. Use Non Saturating Activation Functions: Such as ReLu and its variant.
3. Apply Batch Normalization



Batch Normalization

```
from tensorflow.keras.layers import Dense,  
BatchNormalization
```

```
x = Dense(128)(input)
```

```
x = BatchNormalization()(x)
```

```
x = Activation("relu")(x)
```

Simple analogy

Imagine you're learning in a classroom:

- If the teacher keeps changing the difficulty level suddenly → you struggle.
- BatchNorm keeps the difficulty consistent, so learning becomes smoother.

Drawbacks of simple RNNs

We've now seen the architecture of a standard recurrent neural network. Whilst they are big improvement in terms of dealing with word order, they do have some shortcomings in their current state. These include:

- **Struggling to capture long range dependencies** because the signal gets weakens over time. Simple RNNs have no way of deciding what information is important to 'remember' and what can be 'forgotten'.
- **Vanishing and/or exploding gradients** can occur during training because of the many components affecting each matrix of weights.

There are some ways to dampen the effects of these problems, like gradient clipping, batch normalization, weight initialization etc. However, better cell architectures have been found, which avoid these problems and produce better results. These include GRU and LSTM, which we will discuss now.

Drawbacks of Simple RNNs

- Struggle to capture long-range dependencies as signals weaken over time.
- Cannot decide which information to remember or forget.
- Vanishing or exploding gradients can occur during training.
- Mitigation strategies include gradient clipping, batch normalization, and weight initialization.
- Better architectures like GRU and LSTM overcome these issues.

Parameters in SimpleRNN

$\text{Params} = (\text{input_dim} \times \text{units}) + (\text{units} \times \text{units}) + \text{units}$

```
model = Sequential([
    SimpleRNN(3),
    Dense(1)
])
Model: "sequential"
```

Layer (type)	Output Shape	Param #
simple_rnn (SimpleRNN)	(None, 3)	27
dense (Dense)	(None, 1)	4

Total params: 31 (124.00 B)

Trainable params: 31 (124.00 B)

Non-trainable params: 0 (0.00 B)

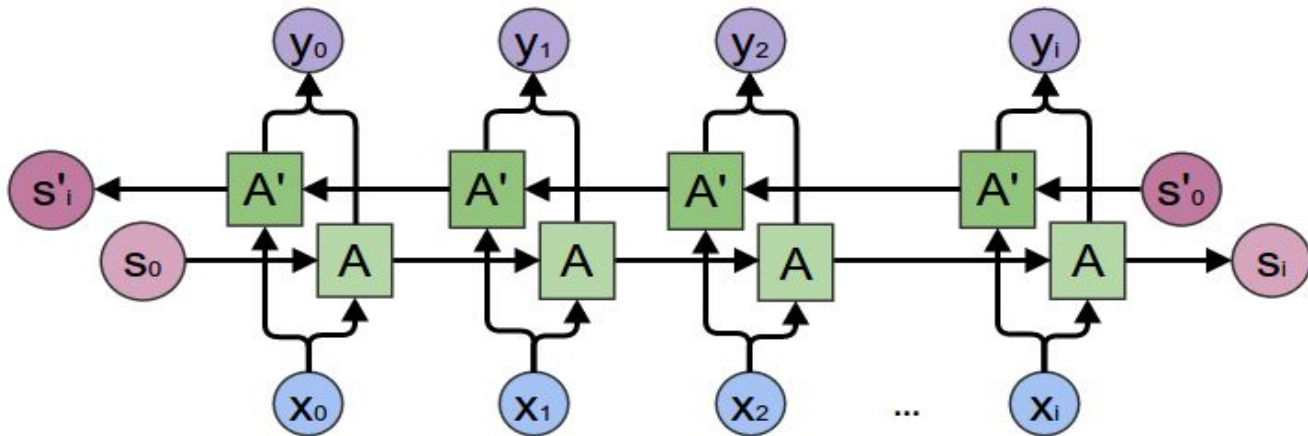
Code Example

<https://colab.research.google.com/drive/1P-ast57G5Admz0we5kNVNL8-PgGMN7R-?usp=sharing>

https://colab.research.google.com/drive/1nP_XzWQhnxYUpQkUlcNsyJCfGhTH3i9S?usp=sharing

RNN Variants

Bidirectional RNNs



Consider the word “bank” in the sentences:

1. “He went to the bank to deposit money.”
2. “The river bank was flooded after the rain.”

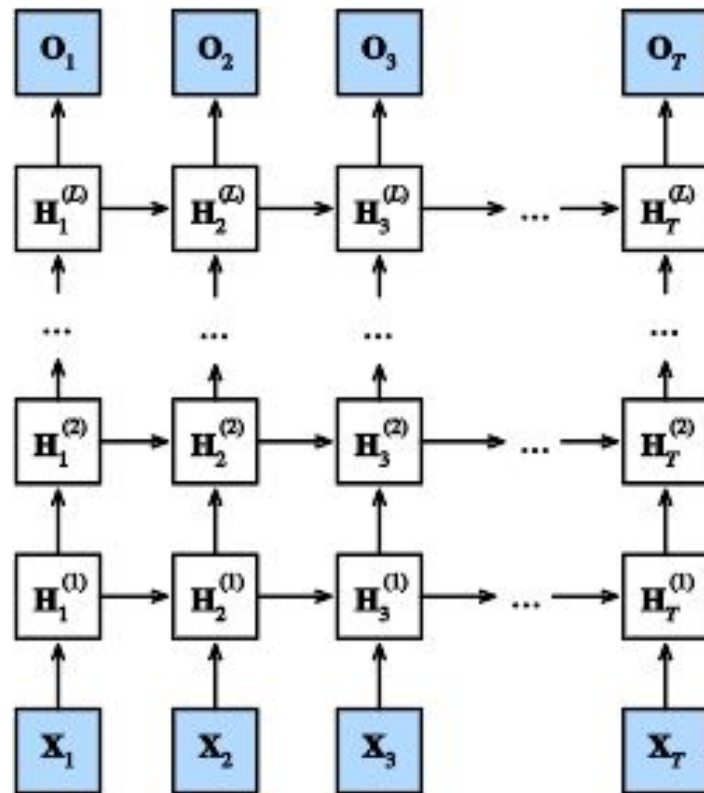
The meaning of “bank” depends on the words that come after it: in the first sentence it refers to a financial institution, in the second it refers to the side of a river.

A bidirectional RNN can capture context from both previous and future words, helping disambiguate the meaning.

Note that the cells in this model can be either simple RNN, GRU or LSTM units depending on your preference. It is even possible to stack bidirectional RNNs to create deep bidirectional RNNs

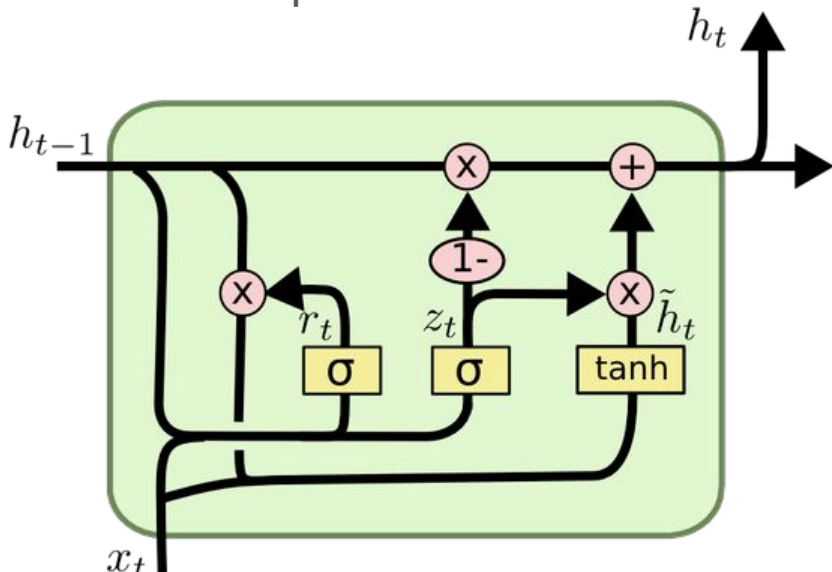
Deep RNNs

- Stack multiple RNN layers to learn more complex representations.
- Each RNN layer outputs a sequence that can feed into the next layer.
- Useful for complex tasks like machine translation.
- Trade-off: Increased depth $\rightarrow$ longer training time.



GRU

GRU stands for Gated Recurrent Unit and is a variant of a RNN where the cell/unit has the ability to forget information it deems irrelevant. As a result, it tends to do a better job at capturing long range dependencies at the cost of increasing the number of parameters in the model.



$$z_t = \sigma (W_z \cdot [h_{t-1}, x_t])$$

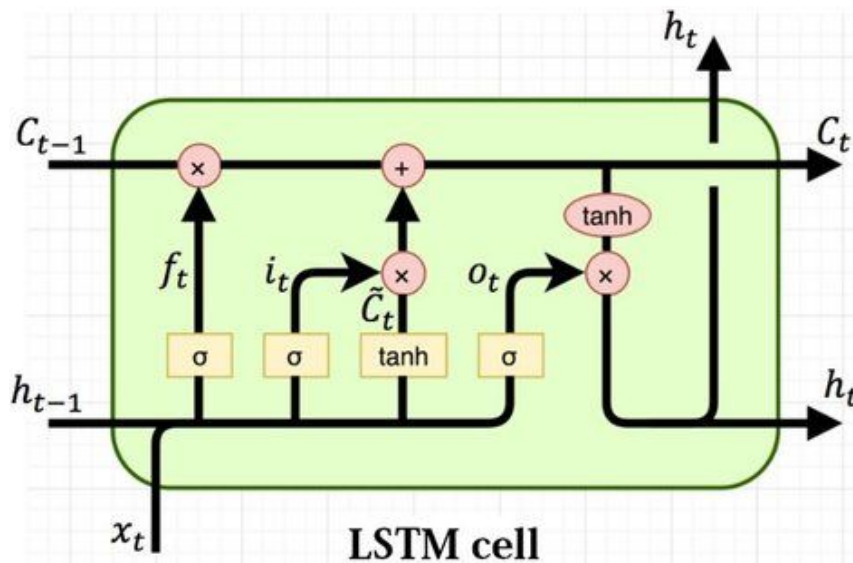
$$r_t = \sigma (W_r \cdot [h_{t-1}, x_t])$$

$$\tilde{h}_t = \tanh (W \cdot [r_t * h_{t-1}, x_t])$$

$$h_t = (1 - z_t) * h_{t-1} + z_t * \tilde{h}_t$$

LSTM

- LSTM stands for Long Short-Term Memory.
- Similar to GRU, but has more parameters and can manage more complex information transfer.
- In addition to the hidden state, LSTM has a cell state that captures long-term memory.
- Allows the model to make predictions using both short-term and long-term memory.
- More complex than GRU, which can lead to better performance on some tasks.
- Trade-off: Increased number of parameters $\rightarrow$ longer training time.



$$i_t = \sigma(x_t U^i + h_{t-1} W^i)$$

$$f_t = \sigma(x_t U^f + h_{t-1} W^f)$$

$$o_t = \sigma(x_t U^o + h_{t-1} W^o)$$

$$\tilde{C}_t = \tanh(x_t U^g + h_{t-1} W^g)$$

$$C_t = \sigma(f_t * C_{t-1} + i_t * \tilde{C}_t)$$

$$h_t = \tanh(C_t) * o_t$$

Create a deep learning model architecture

- "Select keras model type": `sequential`
- "input_shape": `(500,)`
- In "LAYER":
 - ☐ "1: LAYER":
 - "Choose the type of layer": `Embedding -- Embedding`
 - "input_dim": `10000`
 - "output_dim": `32`
 - ☐ "2: LAYER":
 - "Choose the type of layer": `Recurrent -- LSTM`
 - "units": `100`
 - ☐ "3: LAYER":
 - "Choose the type of layer": `Core -- Dense`
 - "units": `1`
 - "Activation function": `sigmoid`
- Click "Run Tool"

Task

Sentiment analysis

- Sentiment classification of IMDB movie reviews with RNN
- Train RNN using IMDB movie reviews
- Goal is to learn a model such that given a review we predict whether review is positive/negative
- We evaluate the trained RNN on test dataset and plot confusion matrix

Assignment#02 is upload on
Teams